

COMP 361D2: Design Document

Progress Report 4

“OpenRobotics”: Humanoid Robot Warehouse Simulator

February 11, 2026

Murad Novruzov, Dan Moraru, Daniel Blackburn,
Lounes Azzoun, Muhammed Sohail,
Badr Zeji, Filip Snítíl, Behnam Yosufi

OpenRobotics

Table of Contents

1. Definitions	4
1.1 Development Cycle	4
1.2 Engineering Procedures	7
1.2.1 Development Workflow & Collaboration	7
1.2.2 Testing & Quality Assurance	7
1.2.3 Database & Configuration Management	8
1.2.4 Documentation Standards	9
1.3 Engineering Constraints and Assumptions	10
1.4 Engineering Success and Failure Statements	14
2. Solution Architecture	20
2.1 Development Environment	20
2.2 Staging Environment	23
2.3 Deployment Environment	25
2.4 Evolving Environment	28
3. Software Design	30
3.1 Storyboard	30
3.1.2 Storyboard Component Descriptions	31
3.2 Code Structure Identification	33
3.2.1 System-Wide Structures	33
3.2.2 Module-Based Structures	34
3.2.3 Procedure-Based Structures	35
Development Phase	35
Deployment / Execution Phase	35
Evolution Phase	36
3.3 UML Domain Model	37
3.3.1 Editor UI Subsystem	37
3.3.2 Simulation Persistence Subsystem	38
3.3.3 Simulation Core Subsystem	38
3.3.4 Warehouse Entities Subsystem	39
3.3.5 Property System	40
3.3.6 Global Classes	40
3.4 Module breakdown	41
3.4.1 Simulation Engine Module	41
3.4.2 Sensors Module	41
3.4.3 Robot Core Module	42
3.4.4 Dispatcher Module	42
3.4.5 Algorithms Module	43
3.4.6 Map Module	43
3.4.7 Database Client Module	44

3.4.8 UI Module	44
3.4.9 File Loader Module	45
4. Database, File, API Definitions	47
4.1 Database Diagrams	47
Figure 4.1.1 – Database Diagram	48
4.2 Database Schemas	49
4.2.1 maps	49
4.2.2 simulation_runs	49
4.2.3 run_results	50
4.2.4 sim_logs	51
4.2.5 robot_run_stats	52
4.2.6 run_workload_tasks	53
4.3 File Definitions	55
4.3.1 Map Definition Files (JSON)	55
4.3.2 Simulation Configuration Files (JSON)	55
4.3.3 Result Export Files (CSV / JSON)	55
4.3.4 Logging and Temporary Files	56
4.4 API Definitions	57
4.4.1 Map APIs	57
4.4.2 Simulation Run APIs	57
4.4.3 Results and Metrics APIs	57
4.4.4 Logging and Replay APIs	58
4.4.5 Export APIs	58
5. Critical Sections	59
5.1 Risk Analysis	59
5.2 Mitigation Models	60
Figure 5.2.1 – CS1: Tick Order	60
Figure 5.2.2 – CS2: Move Intentions	61
Figure 5.2.3 – CS3: Task Assignment	63
Figure 5.2.4 – CS4: Traffic Rules Coordination Policy	64
Figure 5.2.5 – CS5: Tile Reservation	65
Figure 5.2.6 – CS6: Deadlock Detection	66
Figure 5.2.7 – CS7: Greedy Strategy	67
Appendix A: Teams Contribution Table	68

1. Definitions

1.1 Development Cycle

The OpenRobotics project follows a 14-week development timeline spanning from January 5 to April 14, 2026, organized into seven two-week sprints. This structured approach allows the eight-student team to systematically progress from initial specification through final implementation while maintaining clear milestones and deliverables aligned with COMP361 course requirements.

Hybrid Development Methodology

The project employs a hybrid methodology that combines elements of both Waterfall and Agile approaches, tailored specifically to the constraints and expectations of the course. The initial phases follow a Waterfall model for foundational design work:

Specification Phase: Completed by January 21, 2026, establishing core requirements, including must-have features such as pathfinding algorithms, coordination policies, and collision/deadlock detection systems.

Design Phase: Due February 11, 2026, defining the architectural blueprint for the deterministic Java/JavaFX architecture with PostgreSQL database integration. Following the completion of design documentation, the team transitions to Agile practices for implementation.

This approach embodies the course principle of "Capturing the Big Ideas ~ Agile the Details," where major architectural decisions and system design are established upfront, while feature implementation proceeds iteratively with flexibility to adapt based on feedback and technical discoveries.

Sprint Structure and Iterative Build Approach

Each two-week sprint is structured around incremental progress toward a working warehouse simulator. The team follows an iterative build strategy that prioritizes establishing the complete system skeleton early, then progressively populating functionality across all modules. This means:

- Sprint 1-2 (Jan 5 - Feb 1): Foundation establishment, including project setup, core data models, basic UI framework, and database schema implementation.
- Sprint 3-4 (Feb 2 - Mar 1): Core algorithm implementation and integration, focusing on pathfinding algorithms and basic coordination mechanisms. Simple GUI, with minimal interaction between frontend and backend.
- Sprint 5 (Mar 2 - Mar 15): Feature completion for Demo 1 (March 13), showcasing functional robot navigation and warehouse operations. Detailed GUI with more features.

- Sprint 6 (Mar 16 - Mar 29): Advanced features including TRAFFIC_RULES and RESERVATION_K coordination policies, and ensuring simulation metrics logged and formatted properly on frontend. Preparing for Demo 2 (March 31).
- Sprint 7 (Mar 30 - Apr 14): Polish, optimization, bug fixes, and final integration ahead of the final submission (April 14).

This incremental approach ensures that all major components of OpenRobotics evolve in parallel rather than completing one module entirely before moving to the next, reducing integration risks and enabling earlier detection of architectural issues.

Custom Workflow Process

The team follows a mixed workflow adapted to the course's collaborative environment:

- **Sprint Planning:** At the beginning of each two-week sprint, the team meets to define sprint goals, identify specific features or components to implement, and assign tasks using transparent tracking systems.
- **Daily/Asynchronous Updates:** Team members provide progress updates through asynchronous communication channels, maintaining awareness of blockers, completed work, and dependencies without requiring synchronous daily meetings.
- **Sprint Reviews with Demos:** At the conclusion of each sprint, the team conducts a review session where working features are demonstrated. This practice directly prepares the team for the formal Demo 1 (March 13) and Demo 2 (March 31) presentations.
- **Retrospectives:** Following each sprint review, the team holds a retrospective to discuss what worked well, what challenges arose, and how the development process can be improved for subsequent sprints.

Key Milestones

The development cycle is anchored by critical course milestones that ensure steady progress and accountability:

- **Specification Submission:** January 21, 2026 (Completed)
- **Design Document Submission:** February 11, 2026
- **Demo 1:** March 13, 2026 (Mid-project demonstration of core functionality)
- **Demo 2:** March 31, 2026 (Demonstration of advanced features and polish)
- **Final Submission:** April 14, 2026 (Complete system delivery)
- **Acceptance Testing:** April 16-30, 2026 (Validation against requirements)
- **Version Control and Collaboration**

All code development is managed through GitHub using a feature-branch workflow. Each new feature or bug fix is developed in an isolated branch, allowing team members to work independently without interfering with the main codebase. Before any code is merged into the main branch, it undergoes peer review through GitHub Pull Requests (PRs), ensuring code

quality, catching potential issues early, and facilitating knowledge sharing across the team. This disciplined approach to version control is essential for coordinating work across eight developers and maintaining a stable, production-ready codebase throughout the 14-week timeline.

1.2 Engineering Procedures

This section outlines the standardized engineering procedures that govern the OpenRobotics project development lifecycle. These procedures ensure consistent quality, effective collaboration among the 8-member student team, and maintainability of the codebase.

1.2.1 Development Workflow & Collaboration

Team Communication: The team maintains weekly synchronous meetings for sprint planning, blocker resolution, and demo progress review. Asynchronous communication channels support daily status updates and issue tracking, with all tasks, bugs, and features tracked using labels for categorization (enhancement, bug, documentation, testing, database). Sprint management uses a Kanban board workflow with clear status tracking from backlog through completion.

Code Review Process: All pull requests require at least one approval from another team member before merging. Reviews verify code quality, test coverage, documentation completeness, and adherence to style guidelines, with reviewers committing to timely reviews to maintain development velocity.

Feature Branch Workflow: Developers pull the latest changes from the main branch daily, create descriptive feature branches with issue number references, implement features with incremental commits, and test locally before pushing. Pull requests include detailed descriptions, issue references, and testing documentation. Upon approval, changes are merged to the main branch, and feature branches are deleted to maintain repository hygiene.

Main Branch Protection: The main branch must always build successfully and remain deployment-ready for demos and milestones. Branch protection rules enforce PR reviews and passing automated checks before merging.

1.2.2 Testing & Quality Assurance

Unit Testing: The project uses JUnit 5 (JUnit Jupiter 5.10.1) for backend testing and TestFX 4.0.18 for automated JavaFX UI testing, with a minimum 80% code coverage required for core logic modules (Robot, Warehouse, Task, Pathfinding). Tests mirror the source structure and execute automatically via *mvn test* before every push. Test naming follows standard conventions for clarity and maintainability.

Integration & System Testing: Integration tests validate interactions between major components (Robot-Warehouse-Database pipeline) and are conducted before sprint demos. End-to-end simulation runs with predefined configurations verify complete system functionality. The staging environment supports pre-deployment validation of all features before milestone demos.

Determinism Testing: Automated tests verify that identical configuration files produce identical simulation outcomes across multiple runs. Random number generators use fixed seeds for reproducible test scenarios, ensuring the deterministic behavior required for simulation validity.

Performance Testing: The system is tested with varying map sizes (5x5, 10x10, 20x20 grids) and robot counts (1, 5, 10, 20 robots) to verify acceptable performance thresholds. Metrics track simulation step execution time, memory usage, and database query latency to identify potential performance degradation.

Bug Management: All bugs are reported as tracked issues with severity labels (critical, major, minor). Critical bugs blocking core functionality are fixed within the same sprint. Major bugs are scheduled for the next sprint, while minor bugs are backlogged. All bug fixes include regression tests to prevent recurrence.

1.2.3 Database & Configuration Management

Database Schema Management: Schema changes are defined as versioned migration scripts stored in the codebase. Migration naming follows a structured convention, and all schema changes require review by the database-responsible team member before merging. Maven supports automated migration execution.

Environment Separation: The project maintains separate database instances for development and staging to ensure data isolation. Connection credentials are stored in environment variables and never committed to the repository. Pre-migration backups are created before applying any destructive schema changes.

Test Data Management: SQL seeding scripts provide reproducible test datasets, including minimal configurations for rapid testing and comprehensive datasets for integration validation. Database connection pooling manages connections efficiently while respecting service tier limitations.

Configuration Files: Simulation parameters are defined in JSON configuration files with hierarchical organization. JSON schema validation ensures configuration correctness at runtime. Sample configurations support common scenarios (default, performance testing, demos), and all sensitive credentials are managed through environment variables rather than configuration files.

Configuration Loading: The system implements an override hierarchy where command-line arguments take precedence over environment variables, which take precedence over configuration files, which take precedence over hard-coded defaults. Fail-fast validation at startup ensures required settings are present and valid.

1.2.4 Documentation Standards

Code Documentation: All public classes, methods, and interfaces include Javadoc comments with appropriate parameter, return value, and exception documentation. Package-level documentation files explain the purpose and architecture of each major package. Maven generates HTML documentation from Javadoc comments.

Module Documentation: Each major module directory contains a README file explaining module purpose, key classes and relationships, usage examples, and dependencies. Architecture Decision Records (ADRs) document significant technical decisions with context, decision rationale, consequences, and alternatives considered.

User & Developer Guides: Installation guides document Maven setup, framework dependencies, and database configuration. User manuals explain how to run simulations and interpret results, while configuration guides detail all available options. UML diagrams for major subsystems are maintained alongside the codebase and updated when significant architectural changes occur.

Documentation Review: Pull request checklists enforce documentation updates for any public API changes. Documentation is reviewed for clarity, completeness, accuracy, and proper formatting as part of the standard code review process.

1.3 Engineering Constraints and Assumptions

Constraints

Budget Limitations

- Constraint: Zero budget; restricted to free-tier services only
- Expected Impact: Cannot use premium database hosting, monitoring tools, or cloud deployment services; limited to free-tier database storage and connection pooling restrictions
- Workaround Plan: Optimize database schema for minimal storage; implement aggressive data cleanup strategies; use local database for testing; leverage free-tier CI/CD tools

Timeline Pressures

- Constraint: 14-week semester with fixed milestones; Design Document due February 11, 2026; final delivery by the end of the semester
- Expected Impact: Extremely tight deadlines limit time for iteration and refactoring; compressed testing phases
- Workaround Plan: Prioritize core features using the MoSCoW method; use iterative development with 2-week sprints; front-load architecture decisions; prepare boilerplate code and templates in advance

Team Coordination

- Constraint: 8 part-time student developers with competing academic commitments
- Expected Impact: Limited total person-hours available; coordination overhead increases with team size; risk of uneven workload distribution
- Workaround Plan: Establish clear module ownership; use pair programming for complex components; implement a code review process; maintain detailed documentation for knowledge transfer

Experience Level

- Constraint: Team members learning Java and JavaFX during development; limited prior experience with JavaFX UI framework
- Expected Impact: Steeper learning curve impacts velocity; higher risk of architectural mistakes; increased debugging time for UI and concurrency issues
- Workaround Plan: Allocate the first sprint for JavaFX tutorials and prototyping; create a shared code examples repository; schedule knowledge-sharing sessions; assign experienced members as mentors

Database Service Limitations

- Constraint: Database free tier limitations, including storage constraints, connection limits, and potential service restrictions
- Expected Impact: Cannot store large simulation histories; risk of connection exhaustion during development; potential service interruptions
- Workaround Plan: Implement connection pooling in the application; store minimal data (configurations and results only); archive old simulations locally; design for database-optional replay mode using structured logs

Cross-Platform Requirements

- Constraint: Application must run on Windows, macOS, and Linux without platform-specific dependencies
- Expected Impact: Increases testing complexity; requires careful selection of cross-platform libraries; file path handling and JavaFX rendering differences
- Workaround Plan: Use Java NIO for file operations; test on all platforms before merges; containerize development environment; avoid platform-specific features; use JavaFX CSS for consistent UI rendering

Academic Integrity

- Constraint: Must adhere to COMP361 Design Document format and evaluation criteria; subject to academic integrity policies
- Expected Impact: Design decisions must be well-documented and justified; cannot use certain third-party libraries without approval; must demonstrate original work
- Workaround Plan: Follow the provided design document template strictly; cite all external resources; implement core simulation logic from scratch; obtain instructor approval for major libraries

Determinism Requirement

- Constraint: Simulation must produce identical results given the same configuration and seed (required for simulation validity and testing)
- Expected Impact: Limits parallelization strategies; requires careful management of random number generation; complicates multithreading implementation
- Workaround Plan: Use seeded RNG with controlled state; implement deterministic event queue; avoid parallel streams for core simulation logic; document all non-deterministic operations

Assumptions

Team Availability

- Assumption: All 8 team members remain available and engaged throughout the 14-week semester
- Expected Impact: Enables consistent sprint planning and even workload distribution; supports parallel development streams
- Workaround Plan: Track attendance and contribution metrics; cross-train on critical modules; maintain backlog buffer; redistribute tasks during sprint planning if members are unavailable

Java Knowledge

- Assumption: Team members possess basic Java programming skills (OOP, collections, exception handling)
- Expected Impact: Allows immediate focus on JavaFX and architecture rather than language fundamentals; supports reasonable sprint velocity estimates
- Workaround Plan: Conduct skill assessment in Week 1; provide Java refresher materials; pair less experienced members with experienced developers; allocate extra time for code reviews

Database Reliability

- Assumption: Database hosting provides sufficient uptime and performance for development and demonstration purposes
- Expected Impact: Enables cloud-based collaboration on shared database; supports multi-user testing scenarios; provides realistic deployment environment
- Workaround Plan: Implement local PostgreSQL fallback for development; export database schema regularly; design application to gracefully handle database unavailability; cache critical data locally

JavaFX Cross-Platform Consistency

- Assumption: JavaFX renders consistently across Windows, macOS, and Linux with Java 21+
- Expected Impact: Simplifies UI development with a single codebase; reduces platform-specific testing burden
- Workaround Plan: Test UI on all target platforms bi-weekly; avoid platform-specific CSS; use JavaFX layout managers instead of absolute positioning; document known rendering differences

Memory Constraints

- Assumption: Application runs smoothly on typical student laptops with 8GB RAM; simulation state fits in heap memory
- Expected Impact: Allows all team members to develop and test locally; supports reasonable warehouse sizes (e.g., 50x50 grid, 100 robots)
- Workaround Plan: Profile memory usage regularly; implement object pooling for frequently created entities; limit simulation scale based on testing; provide memory configuration guidelines

JSON Configuration Sufficiency

- Assumption: JSON format is sufficient for all configuration needs (warehouse layout, robot parameters, item distributions)
- Expected Impact: Simplifies configuration file creation and editing; enables easy version control and human readability
- Workaround Plan: Define comprehensive JSON schema with validation; provide example configurations and schema documentation; implement robust error handling for malformed JSON

Deterministic Execution

- Assumption: Java execution is deterministic when controlling randomness and avoiding system clock dependencies
- Expected Impact: Enables reproducible simulations for testing and validation; simplifies debugging by eliminating timing-based bugs
- Workaround Plan: Use seeded random number generators with explicit seeds; avoid system time dependencies in simulation logic; implement centralized time management; thoroughly test replay functionality

Network Stability

- Assumption: Internet connectivity is stable enough for database access during development and demonstrations
- Expected Impact: Enables real-time collaboration on shared database; supports cloud-based configuration storage and result archival
- Workaround Plan: Implement retry logic with exponential backoff; cache database queries locally; provide offline development mode; test demo scenarios with pre-loaded data

1.4 Engineering Success and Failure Statements

Success Criteria

Feature Completeness

- Success Statement: All required navigation algorithms and both coordination policies (TRAFFIC_RULES, RESERVATION_K) are fully implemented and functional. Collision and deadlock detection systems operate correctly across all scenarios.
- Expected Impact: Ensures the project meets course acceptance criteria and provides a complete warehouse simulation system. Demonstrates mastery of core algorithmic concepts.
- Discussion & Alternatives: All features are mandatory course requirements with no viable alternatives. Implementing a subset first risks failing acceptance criteria.

On-Time Delivery

- Success Statement: Design document submitted by February 11, 2026. All milestones (implementation, testing, final presentation) were met according to the course schedule.
- Expected Impact: Maintains project timeline and avoids grade penalties. Allows adequate time for iterative testing and refinement before final submission.
- Discussion & Alternatives: Extensions may impact grades and compress downstream milestones. Time management and parallel workstreams are critical.

Deterministic Behavior

- Success Statement: Given identical configuration files, the simulator produces bit-exact identical results across multiple runs (same paths, same collision events, same metrics).
- Expected Impact: Essential for validation and grading. Enables reproducible debugging and demonstrates understanding of deterministic simulation principles.
- Discussion & Alternatives: Allowing minor floating-point variations would complicate validation. Strict determinism is preferred and achievable with careful design.

Code Quality

- Success Statement: Codebase maintains consistent style, comprehensive Javadoc documentation for all public APIs, and follows SOLID principles. Code review process catches issues before merge.
- Expected Impact: Facilitates team collaboration, simplifies maintenance, and demonstrates professional software engineering practices. Makes debugging and future extensions straightforward.

- Discussion & Alternatives: Minimal documentation saves time initially, but harms long-term maintainability and team efficiency. Upfront investment in quality pays dividends.

Team Collaboration

- Success Statement: Clear ownership of modules, regular sync meetings, effective use of version control (branching strategy, pull requests), and transparent communication channels.
- Expected Impact: Prevents duplicate work, reduces integration conflicts, and ensures all team members understand system architecture. Improves overall team velocity.
- Discussion & Alternatives: Ad-hoc coordination increases the risk of miscommunication and wasted effort. Structured collaboration is essential for multi-person projects.

Performance

- Success Statement: Typical warehouse scenarios (20+ robots, 100x100 grid, 500 tasks) complete simulation in under 5 minutes on standard hardware. UI remains responsive during execution.
- Expected Impact: Enables iterative testing and validates scalability. Demonstrates efficient algorithm implementation and proper use of data structures.
- Discussion & Alternatives: Accepting slower performance hinders development workflow and may indicate algorithmic inefficiencies requiring optimization.

Database Reliability

- Success Statement: All simulation metrics (paths taken, collisions, completion times, coordination events) are correctly persisted to the database. Zero data loss or corruption. Queries return accurate results for analysis.
- Expected Impact: Provides a verifiable audit trail for grading. Enables post-simulation analysis and performance comparison across configurations.
- Discussion & Alternatives: File-based logging complicates querying and analysis. A database solution provides robustness and querying power required for metrics.

UI Usability

- Success Statement: JavaFX interface allows users to load configurations, start/pause/stop simulations, visualize robot movements in real-time, and inspect metrics without confusion. Controls are intuitive and responsive.
- Expected Impact: Enhances demonstration quality and user experience. Simplifies testing and debugging by providing a clear visualization of the simulator state.
- Discussion & Alternatives: Command-line only interface limits demonstration effectiveness and makes debugging harder. GUI is worth the implementation investment.

Test Coverage

- Success Statement: Unit tests cover core algorithms (navigation, coordination, collision detection) with >80% code coverage. Integration tests validate end-to-end scenarios.
- Expected Impact: Catches regressions early, ensures correctness, and provides confidence in system behavior. Demonstrates rigorous testing methodology.
- Discussion & Alternatives: Manual testing only is time-consuming, error-prone, and doesn't scale. Automated testing is essential for complex systems.

Acceptance Criteria Pass

- Success Statement: Project passes all course-defined acceptance tests on first submission without requiring corrections or resubmission.
- Expected Impact: Validates that all requirements are met and demonstrates a thorough understanding of project specifications. Avoids grade penalties.
- Discussion & Alternatives: No alternative—passing acceptance criteria is mandatory for project success.

Failure Criteria (Harbingers of Failure)

Missed Deadlines

- Failure Statement: Design document submitted after February 11, 2026, or subsequent milestones (implementation, testing, final presentation) are delayed.
- Expected Impact: Grade penalties, compressed testing time, increased stress, and cascading delays for downstream milestones. May result in incomplete or rushed implementation.
- Discussion & Alternatives: Mitigation requires buffer time in the schedule, starting work immediately, tracking progress weekly, and escalating blockers early.

Integration Failures

- Failure Statement: JavaFX frontend cannot communicate with the backend simulation engine, or the database layer fails to integrate with core logic. Components work in isolation but not together.
- Expected Impact: System is non-functional end-to-end. Requires extensive rework and debugging. Delays delivery and may force emergency refactoring under time pressure.
- Discussion & Alternatives: Mitigation requires defining clear interfaces early, implementing integration tests from sprint 1, and using dependency injection to decouple components.

Non-Deterministic Behavior

- Failure Statement: Same configuration produces different simulation results across runs due to race conditions, uninitialized variables, or non-deterministic data structures (e.g., HashMap iteration order).
- Expected Impact: Fails acceptance criteria. Makes debugging impossible. Invalidates all metrics and testing. Demonstrates a lack of understanding of deterministic simulation requirements.
- Discussion & Alternatives: Mitigation requires using deterministic collections, avoiding multithreading in the simulation core, seeding random number generators, and conducting early determinism validation tests.

Poor Documentation

- Failure Statement: Code lacks Javadoc, architecture decisions are undocumented, or README files are incomplete/missing. Team members struggle to understand each other's code.
- Expected Impact: Slows development, increases onboarding time, complicates debugging, and harms code review quality. May lose points on the code quality rubric.

- Discussion & Alternatives: Mitigation requires enforcing documentation standards in pull request checklists, allocating time for documentation in sprint planning, and using templates for module documentation.

Database Issues

- Failure Statement: Database connection failures, incorrect SQL queries, data loss during simulation, or metrics are recorded inconsistently. The database schema doesn't match the application needs.
- Expected Impact: Metrics are unreliable or missing, breaking analysis and grading requirements. May require database migration or redesign under time pressure.
- Discussion & Alternatives: Mitigation requires early database integration testing, connection pooling, transaction safety, schema validation against requirements, and rollback planning.

Performance Problems

- Failure Statement: Simulations take excessive time for typical scenarios, UI freezes during execution, or system crashes with large warehouse configurations (memory leaks, stack overflow).
- Expected Impact: Makes iterative testing infeasible. Demonstrates inefficient algorithm implementation. May fail acceptance criteria if performance requirements are specified.
- Discussion & Alternatives: Mitigation requires profiling code early, using appropriate data structures, avoiding premature optimization while watching for obvious inefficiencies, and testing with large scenarios incrementally.

Team Conflicts

- Failure Statement: Communication breakdown, unclear module ownership, merge conflicts due to poor branching strategy, or team members working in silos without coordination.
- Expected Impact: Duplicate work, integration delays, low morale, and uneven workload distribution. May result in finger-pointing and missed deadlines.
- Discussion & Alternatives: Mitigation requires establishing clear roles, holding regular standups, using task tracking systems, defining Git workflow upfront, and addressing conflicts proactively.

Scope Creep

- Failure Statement: Team invests significant time in nice-to-have features (e.g., advanced analytics dashboards) before completing must-have features.
- Expected Impact: Must-have features remain incomplete or buggy. Fails acceptance criteria. Wastes time that should be spent on core requirements.
- Discussion & Alternatives: Mitigation requires strict prioritization, deferring all nice-to-haves until after acceptance criteria pass, conducting weekly scope reviews, and empowering the tech lead to veto non-critical work.

Inadequate Testing

- Failure Statement: Unit tests are missing or superficial, edge cases are untested, or integration tests are added only at the end. Bugs are discovered late in the development cycle.
- Expected Impact: Major bugs found during acceptance testing require emergency fixes. May fail acceptance criteria or deliver an unstable system. Last-minute debugging introduces new bugs.
- Discussion & Alternatives: Mitigation requires writing tests alongside implementation, automating test execution in CI/CD, allocating a dedicated testing sprint before the deadline, and prioritizing high-risk areas.

Dependency Failures

- Failure Statement: External dependencies (JavaFX libraries) have breaking changes, network connectivity issues during demo, or the team lacks access to required resources.
- Expected Impact: System is non-functional during critical moments (demo, grading). May require hasty workarounds or alternative solutions.
- Discussion & Alternatives: Mitigation requires locking dependency versions, testing with fallback configurations, maintaining a local database for testing, documenting all setup requirements, and having a contingency plan for demos.

2. Solution Architecture

2.1 Development Environment

The development environment provides a local workspace where each developer builds, tests, and debugs the OpenRobotics warehouse simulator.

IDE and SDK

The team uses IntelliJ IDEA as the primary IDE, configured with the Eclipse Temurin JDK 21 (temurin-21). IntelliJ provides integrated Maven support, JavaFX scene builder compatibility, Git version control, and a built-in debugger for stepping through simulation tick logic. Developers launch the application directly from IntelliJ using the Maven `javafx:run` goal.

Build System

Maven manages compilation, dependency resolution, testing, and packaging. The project is defined as `com.openrobotics:open-robotics:1.0.0` with the following key plugins:

- `maven-compiler-plugin 3.11.0` — compiles Java 21 source code
- `javafx-maven-plugin 0.0.8` — launches the JavaFX application (main class: `com.openrobotics.MainApp`)
- `maven-surefire-plugin 3.2.5` — executes JUnit 5 and TestFX test suites

Standard build commands used by the team:

- `mvn clean javafx:run` — clean build and launch the simulator
- `mvn test` — run all unit and UI tests
- `mvn package` — produce the distributable JAR
- `mvn clean compile` — compile without running

Dependencies

Maven resolves the following dependencies automatically:

- `JavaFX Controls 21` and `JavaFX FXML 21` — UI components and FXML layout loading
- `PostgreSQL JDBC Driver 42.7.7` — database connectivity
- `JUnit Jupiter 5.10.1` (`junit-jupiter-api`, `junit-jupiter-engine`) — backend unit testing
- `TestFX 4.0.18` (`testfx-core`, `testfx-junit5`) — automated JavaFX UI testing

Database Setup

Developers connect to either a local PostgreSQL instance or the shared Supabase development database. The schema consists of six tables (`maps`, `simulation_runs`, `run_results`,

sim_logs, robot_run_stats, run_workload_tasks). Connection parameters are configured via environment variables to avoid hardcoded credentials. Schema initialization uses versioned migration scripts stored in the codebase.

Configuration Files

JSON configuration files stored in `./config/` define simulation parameters:

- Map definition files specify warehouse layouts (grid dimensions, obstacles, charging stations, pickup/dropoff zones) with preset files for small baseline, narrow aisles, and many intersections scenarios.
- Simulation configuration files control pathfinding algorithm selection, coordination policy (TRAFFIC_RULES or RESERVATION_K), robot count, task generation parameters, and energy model settings.

Configuration validation occurs at startup to catch errors before simulation begins.

Development Workflow

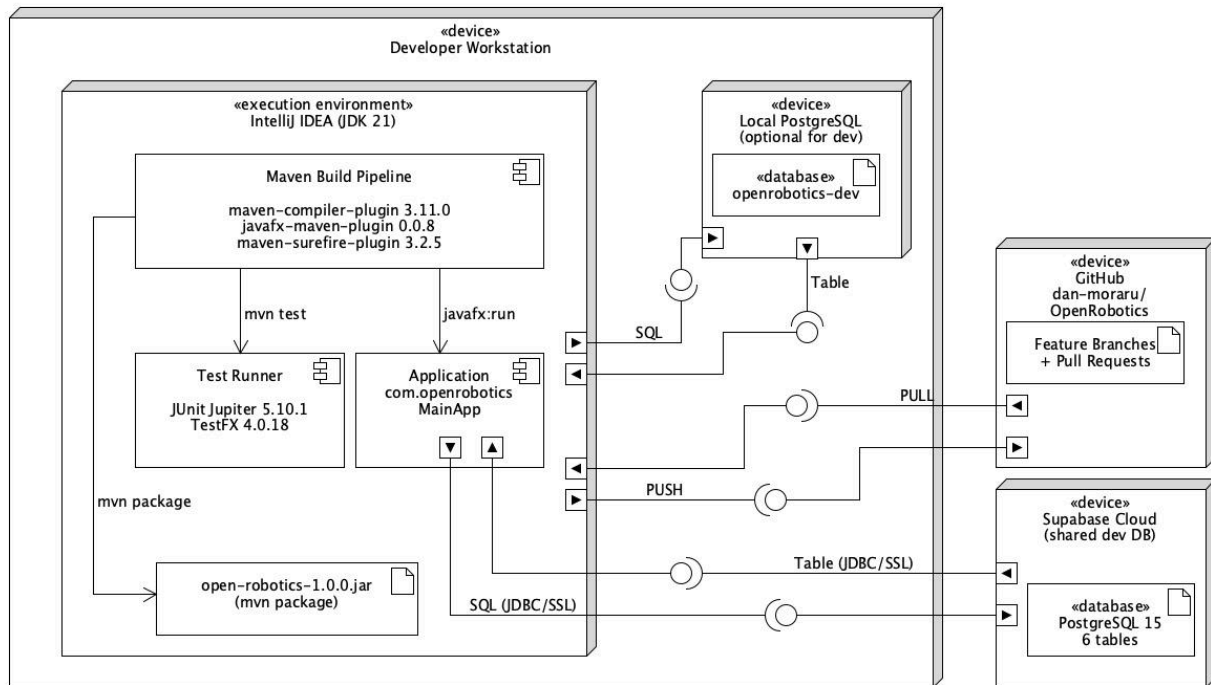
The team follows a Git-based feature-branch workflow hosted on GitHub (github.com/dan-moraru/OpenRobotics):

- Pull the latest changes from main branch
- Create feature branch (e.g., feature/improved-pathfinding)
- Develop with incremental commits; write JUnit 5 tests alongside implementation
- Run mvn test locally to verify correctness before pushing
- Create a pull request with description and testing documentation
- Merge to main after peer approval; delete feature branch

Logging and Debugging

Application logs capture initialization events, tick-by-tick execution details, and error messages. IntelliJ's debugger allows breakpoints and variable inspection during simulation runs, which is particularly useful for tracing pathfinding logic and diagnosing collision avoidance behavior.

Figure 2.1.1 – Development Environment



2.2 Staging Environment

The staging environment is the runtime environment that simulates deployment so QA and the client can validate features safely before demos.

Purpose and Timing

Staging validation occurs before each major milestone:

- One week before Demo 1 (March 13, 2026): core navigation and warehouse operations
- One week before Demo 2 (March 31, 2026): TRAFFIC_RULES and RESERVATION_K coordination policies
- Before Final Submission (April 14, 2026): full system regression

Environment Configuration

Staging uses a dedicated Supabase database instance (openrobotics-staging) isolated from the development database. The staging runtime mirrors deployment (JRE 21+, open-robotics-X.X.X.jar, same config layout and preset maps). The only intended difference is staging data so test datasets can be reset between runs. The staging branch in Git mirrors production-equivalent configuration; developers deploy to staging only after code review approval on their pull request. Because staging is not identical to deployment, configuration, or infrastructure differences may still cause late-stage bugs.

Testing Protocol

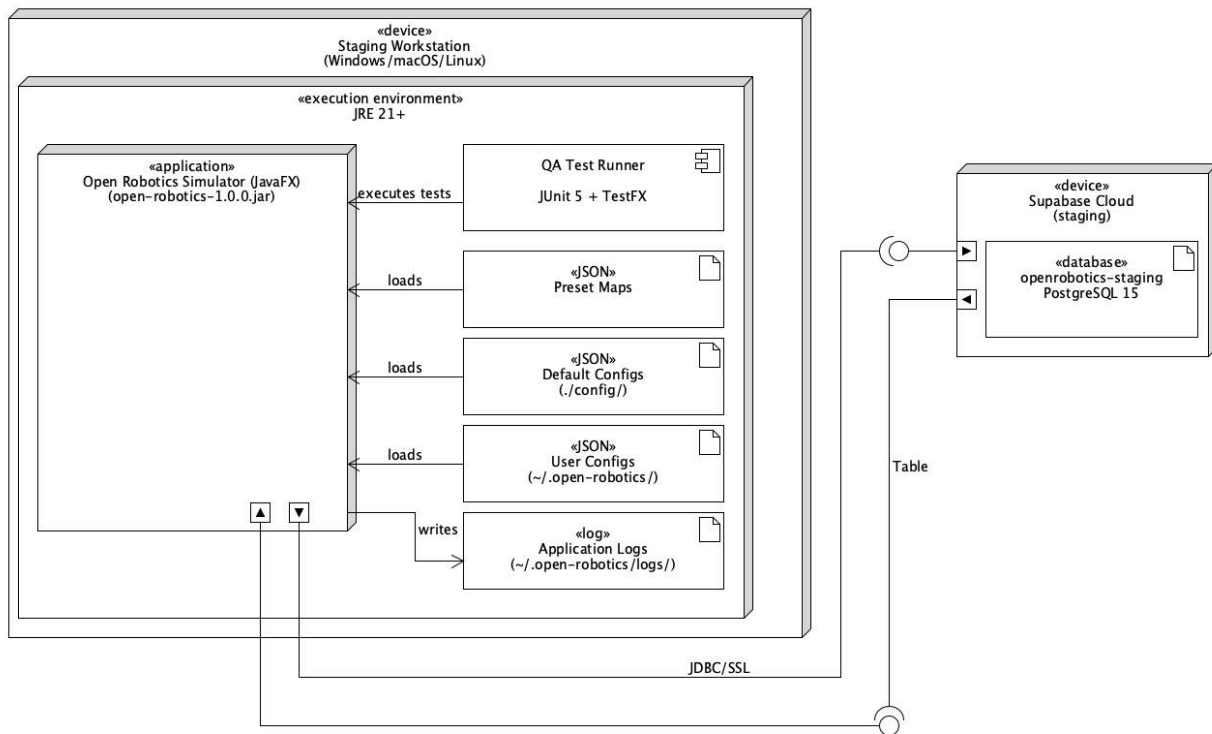
The staging test suite includes:

- JUnit 5 backend tests — validate pathfinding, task assignment, collision detection, and database operations
- TestFX UI tests — verify map loading, simulation controls (start/pause/resume), real-time statistics display, and error handling
- Determinism validation — execute identical configurations with fixed random seeds multiple times, comparing results for bit-exact consistency
- Performance benchmarks — systematic testing across preset maps (small baseline, narrow aisles, many intersections) with varying robot counts (1–20), measuring tick execution time and database query latency
- Database migration testing — verify schema changes execute cleanly with data integrity preserved and rollback procedures tested

Promotion Flow

1. Feature branch development and local tests.
2. Pull request review.
3. Merge to the staging branch and deploy to the staging runtime.
4. Run the staging test suite and demo.
5. Promote to main.

Figure 2.2.1 – Staging Environment



2.3 Deployment Environment

The OpenRobotics simulator is deployed as a standalone desktop application distributed as an executable JAR file.

Client Requirements

The simulator runs on Windows 10/11, macOS 10.15+, and Linux (kernel 4.15+). Minimum hardware: 4GB RAM, dual-core CPU (2.0 GHz+), 500MB disk space, and graphics support for JavaFX rendering. Internet connectivity is required for database access; offline mode falls back to local file logging.

Java Runtime Environment

JRE 21+ (Long-Term Support) is required. The application verifies the JRE version at startup and displays an error if incompatible.

Application Package

Distributed as open-robotics-X.X.X.jar, produced by mvn package. The JAR contains all compiled classes from the com.openrobotics package. The application entry point is com.openrobotics.MainApp.

Internal Modules

The application consists of eight modules:

1. Simulation Engine — centralized deterministic tick pipeline controlling simulation execution
2. Sensors Module — provides environmental awareness to robots
3. Robot Core — manages robot state, movement, and energy
4. Dispatcher Module — assigns tasks to available robots
5. Algorithms Module — implements pluggable pathfinding algorithms
6. Map Module — loads and manages warehouse grid layouts
7. Database Client — handles all PostgreSQL read/write operations via JDBC
8. UI Module — JavaFX interface for configuration, visualization, and results display

Runtime Libraries

Bundled with the JAR:

- PostgreSQL JDBC Driver 42.7.7 — database connectivity
- JavaFX FXML 21 — UI layout loading from FXML files
- JavaFX Controls 21 — UI components (buttons, tables, input fields)

Configuration and Data

- Default configs (./config/): factory defaults, system-level database settings, example templates
- Preset maps (bundled in JAR resources): small baseline, narrow aisles, many intersections — read-only, consistent across all installations
- User configs (~/.open-robotics/): saved custom simulations, user preferences, customized database settings
- Application logs (~/.open-robotics/logs/): diagnostic logs with automatic rotation
- This separation allows application updates without losing user data.

Database Connectivity

PostgreSQL 15 hosted on Supabase Cloud Server provides centralized data storage. Connection via JDBC over SSL (port 6543) supports persistence of simulation runs, execution logs, per-robot statistics, and workload task history across the six database tables (maps, simulation_runs, run_results, sim_logs, robot_run_stats, run_workload_tasks).

Supabase was chosen because it provides a managed PostgreSQL database with a free tier sufficient for the course project needs, enables all 8 team members to access shared experiment data from different computers, and eliminates the need to host a dedicated database server. The managed service handles backups and availability automatically.

Deployment Procedure (Manual + Scripted)

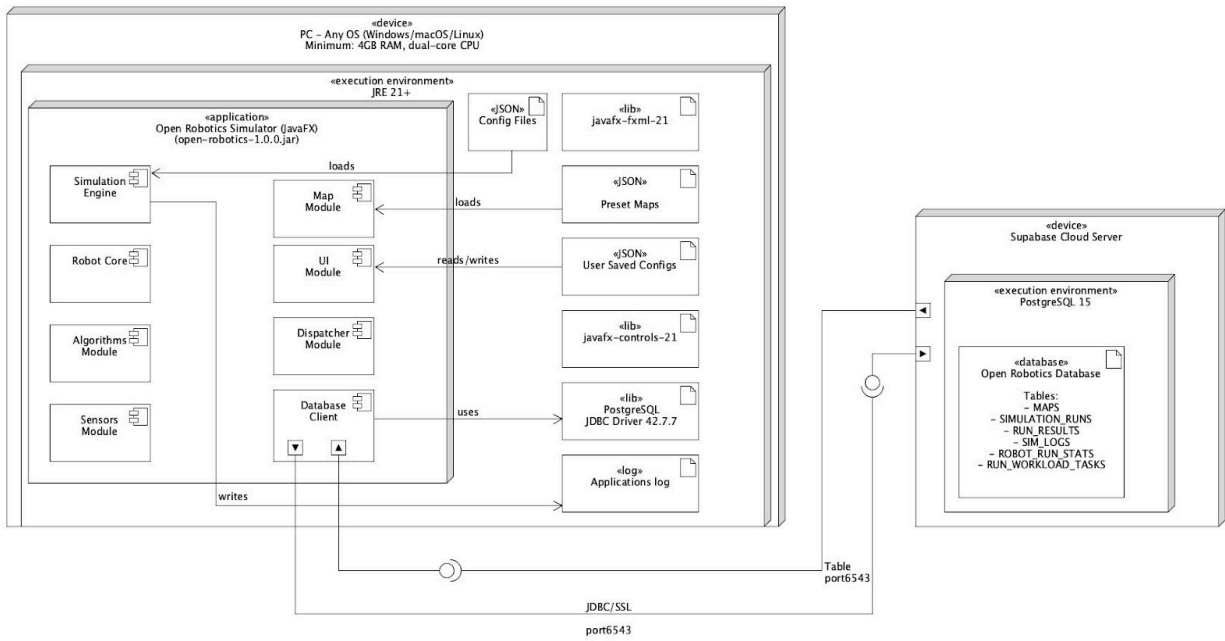
The deployment strategy is a short, repeatable procedure that matches the client runtime environment and minimizes disruption:

1. Verify JRE 21+ and hardware requirements on the target PC.
2. Copy open-robotics-X.X.X.jar to a stable install directory.
3. Create ~/.open-robotics/ and copy default config templates from ./config/.
4. Configure database connection settings via environment variables.
5. Launch java -jar open-robotics-X.X.X.jar and run a smoke test (load preset map, start a short run, verify UI + DB writes).
6. Record installed version and configuration baseline.

Updates follow the same procedure: replace the JAR, preserve ~/.open-robotics/ data, and run the smoke test. Schema changes are applied using migration scripts and verified against staging before rollout.

User workflow: load maps → configure simulation parameters (robot count, coordination policy, RESERVATION_K values) → run simulation with real-time visualization → view metrics → export results to database or local files.

Figure 2.3.1 – Deployment Environment



2.4 Evolving Environment

The evolving environment describes how the system is supported, upgraded, and eventually retired after deployment.

Support and Maintenance

Post-deployment support focuses on defect triage, user assistance, and operational stability:

- Issues are tracked in GitHub with severity labels and reproduction steps.
- Logs in `~/open-robotics/logs/` are used for debugging and regression analysis.
- A short user guide and troubleshooting section document common configuration and runtime errors.
- Support questions are handled in team reviews and demo feedback sessions.

Upgrades and Changes

Changes are introduced through a staged, repeatable update process that protects user data and configuration:

1. Capture new requirements or fixes as GitHub issues and plan them in the sprint backlog.
2. Implement on a feature branch with JUnit 5 and TestFX coverage.
3. Validate in staging using the same runtime configuration as deployment.
4. Release a new JAR and replace the existing `open-robotics-X.X.X.jar`.
5. Preserve `~/open-robotics/` user data and configs across updates.
6. Apply database migration scripts when schema changes are required and verify data integrity.

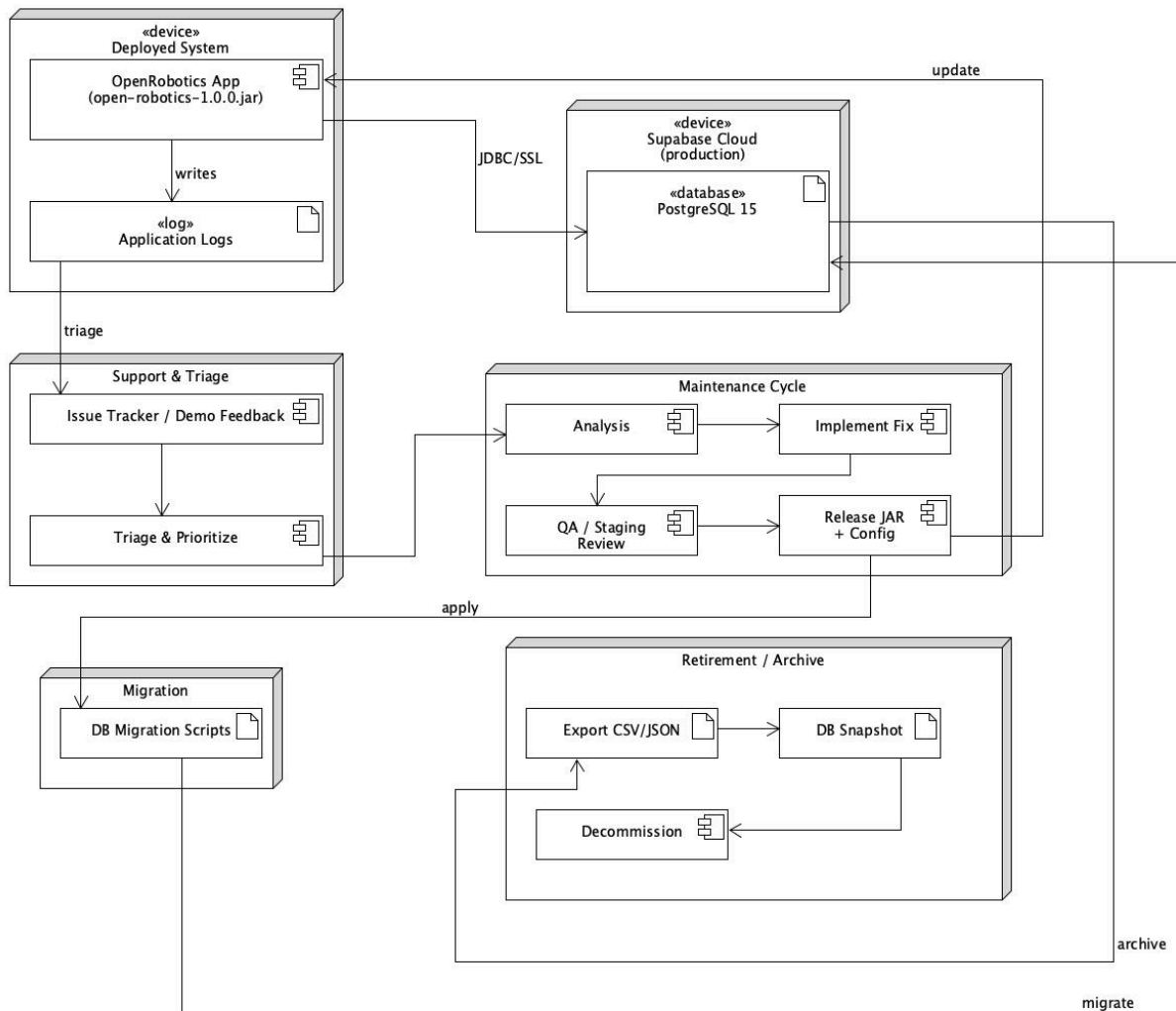
Backwards compatibility is maintained for JSON configs and stored results wherever possible, so older runs remain readable after updates.

Retirement and Data Handling

When the system is retired, data is archived, and the deployment environment is cleaned up:

- Export results and logs to CSV/JSON for long-term reference.
- Archive the Supabase database or take a final snapshot.
- Remove database credentials and decommission the Supabase project.
- Remove the installed JAR and local `~/open-robotics/` directory if required.

Figure 2.4.1 – Evolving Environment



3. Software Design

3.1 Storyboard

Similar storyboard to the Specification document, but with minor improvements and updates. We added more details about file importing and exporting, and a new config file that sets up the backend after creating the map and the database module.

Figure 3.1.1 – Storyboard

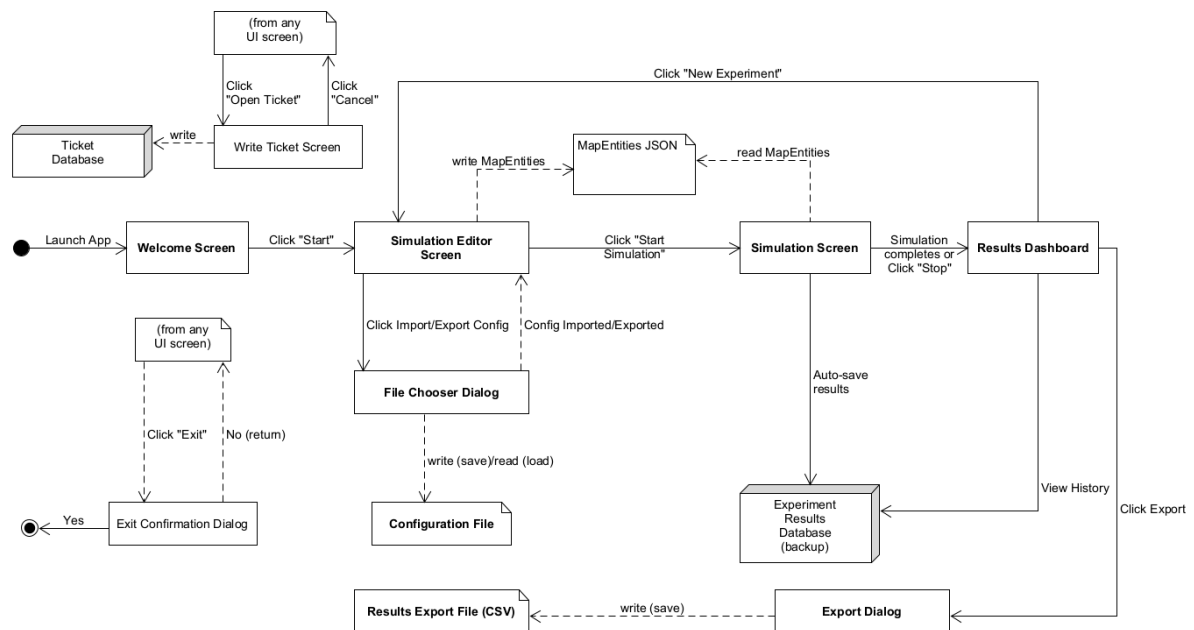


Diagram Key

Symbol	Element Name	Description
Solid Circle	Entry Point	The starting point of the application flow where the user launches the application.
Bullseye Circle	Termination Point	The final end state where the application closes completely.
Rectangle	UI Screen / Dialog	Represents a specific window or dialog box in the application where the user interacts with the system (e.g., <i>Simulation Setup Screen</i>).
3D Box (Node)	Database	Represents the persistent storage system where experiment results and history are stored and retrieved.

Document Icon	Configuration File	Represents external files used to save or load custom simulation settings.
Folded Corner Box	Global Context / Note	Represents a rule or action that applies globally across the application, rather than starting from a single specific screen (e.g., the ability to exit the app from anywhere).
Solid Arrow	Link	Indicates a direct transition from one screen to another, triggered by a user action or system event.
Dashed Arrow	Redirect	Represents a transition that can occur from multiple contexts rather than a direct link from one specific screen.
Labeled Data Arrow	Data Operation	Represents an instantaneous read or write operation between a dialog and an external file or database.

3.1.2 Storyboard Component Descriptions

Welcome Screen

The welcome screen is the first screen users see when launching the application. It displays the application logo and branding, and provides a single entry point into the simulator. The user clicks the "Start" button to proceed to the simulation setup screen. This screen serves as a simple introduction and ensures the application has fully loaded before the user begins configuring their experiment.

Simulation Setup Screen

The simulation setup screen is the central hub where users configure all parameters for their experiment. Users can select or generate a map, specify the number of robots, assign navigation algorithms to each robot, choose a coordination policy, and set workload parameters such as task count and spawn rate. From this screen, users can optionally save their current configuration to a file for later reuse or load a previously saved configuration. Once satisfied with the setup, users click "Start Simulation" to begin the experiment.

File Chooser Dialog and Configuration File

The file chooser dialog appears when users click "Save" or "Load" from the simulation setup screen. When saving, the dialog allows users to specify a filename and location, and the current setup parameters are written to a configuration file instantaneously. When loading, users select an existing configuration file, which is read immediately and populates the setup screen with the

saved values, which they can review before starting. After either operation completes, users are returned to the simulation setup screen. The configuration file is entirely optional and exists for the convenience and reproducibility of experiments.

Simulation Screen

The simulation screen displays the warehouse environment and robots in real time as the experiment runs. Users can observe robots navigating the map, picking up items, delivering to packing stations, and charging when needed. Playback controls allow users to pause, resume, or stop the simulation, as well as adjust the simulation speed (x1, x2, x3, etc.) so users don't have to watch lengthy experiments in real time. The screen also displays live statistics such as tasks completed, active collisions, and elapsed time. The simulation ends either when all tasks are completed, when the maximum tick count is reached, or when the user manually clicks "Stop."

Results Dashboard and Experiment Results Database

The results dashboard presents a summary of the completed experiment, including throughput metrics, collision counts, energy consumption, and per-robot statistics. Results are automatically saved to the database when the simulation ends. Users can also access historical experiment data by clicking "View History," which retrieves past runs from the database for comparison. From this screen, users can click "New Experiment" to return to the simulation setup screen and configure another run.

Export Dialog and Results Export File

The export dialog is triggered when users click "Export" from the results dashboard. This opens the system file explorer, allowing users to select a directory and specify a filename for the exported data. Upon confirmation, the application writes the experiment results to a CSV file at the chosen location. This export includes all metrics from the current experiment, such as throughput, collision counts, energy consumption, and per-robot statistics, formatted for easy analysis in spreadsheet software like Excel. The export feature is separate from the automatic database save and provides users with a portable file they can share or analyze outside the application.

Exit Confirmation Dialog

The exit confirmation dialog can be triggered from any screen in the application by clicking the "Exit" button, represented by the dashed redirect arrow from the global context. This dialog asks users to confirm they want to close the application. Clicking "No" returns users to whatever screen they were previously on, while clicking "Yes" terminates the application and reaches the final end state.

3.2 Code Structure Identification

This section identifies the core technologies, architectural structures, data structures, and procedures used to implement the OpenRobotics system. The code structure is presented through three complementary tables, each capturing a different level of abstraction: system-wide structures, module-based structures, and procedure-based structures. Together, these structures support deterministic simulation execution, modular extensibility, and reproducible experimentation.

3.2.1 System-Wide Structures

System-wide structures represent technologies and architectural elements that apply across the entire application and cannot be attributed to a single module.

Title	Purpose	Technology / Structure	Location	Owner
Programming Language	Core implementation of simulation and backend logic	Java	Entire codebase	Developer
UI Framework	Rendering user interface and user interactions	JavaFX	UI Layer	Developer
Database Engine	Persistent storage of simulation runs, logs, and metrics	PostgreSQL	Persistence Layer	Developer
Configuration Format	Reproducible experiment configuration	JSON files	Configuration loader	Client
Execution Structure – Centralized Simulation Loop	Enforces a fixed per-tick execution order to ensure deterministic and reproducible behavior	Centralized tick pipeline	Simulation Engine	Developer
Logging Mechanism	Support replay, debugging, and analysis	Structured event logging	Logging subsystem	Developer

These structures define the technological and architectural foundation of the system. They enforce global properties such as determinism, persistence, and reproducibility, and therefore apply uniformly across all modules.

3.2.2 Module-Based Structures

Module-based structures correspond to data structures, components, and architectural elements assigned to specific modules or classes.

Title	Purpose	Technology / Structure	Location	Owner
Simulation Engine	Controls per-tick execution, coordinates system components, and enforces coordination policies and collision resolution	Centralized controller	Simulation Engine Class	Developer
Sensors Module	Provides robots with environmental awareness within a configurable sensor range	Sensor view system with partial observability	Sensors Package	Developer
Robot Core	Maintains robot states (position, energy, task, status) and implements state machines and energy models	Collection of robot objects with state tracking	Robot Class	Developer
Dispatcher Module	Manages task generation, assignment, reassignment, and workload queues	Task queues and scheduling logic with deadlock recovery	Dispatcher Class	Developer
Algorithms Module	Implements pluggable navigation strategies (GREEDY, BUG, RTA_STAR) for robot pathfinding	Policy-based navigation algorithms. Uses Strategy Pattern.	Algorithms Package	Developer
Map Module	Stores warehouse layout and tile information	2D grid / tile map	Map class	Client
Database Client	Handles all database operations including logging events, storing metrics, and retrieving historical data	JDBC-based persistence layer. Is a Singleton class.	Database Client Class	Developer
UI Module	Renders JavaFX interface for configuration,	JavaFX controllers and FXML layouts.	UI Package	Developer

	visualization, control, and results display	Inside the Model-Controller-View design pattern		
File Loader	Handles parsing and exporting the simulation instance data. For config files and frontend map creations.	Java, I/O and JavaFX for the frontend. Will probably need the Adapter Pattern.	FileParsing class and JavaFX map creation.	Developer and Client

Each module encapsulates a single responsibility and communicates with other modules through well-defined interfaces. This separation allows navigation algorithms, coordination policies, and persistence mechanisms to evolve independently without impacting the core simulation logic. The Simulation Engine integrates coordination policy enforcement and the CollisionManager component as internal subsystems rather than separate modules, ensuring tight coupling between movement validation and tick execution.

3.2.3 Procedure-Based Structures

Procedure-based structures are organized around phases of the project lifecycle, rather than runtime modules.

Development Phase

Title	Purpose	Technology / Structure	Location	Owner
Unit Testing	Validate correctness of individual modules	JUnit test cases	Test suite	Developer
Logging Inspection	Debug and verify tick-level behavior	Log inspection tools	Development environment	Developer

Deployment / Execution Phase

Title	Purpose	Technology / Structure	Location	Owner
Configuration Loading	Load experiment parameters	JSON parsing	Simulation startup	Client
Tick Execution Pipeline	Advance simulation deterministically	Ordered procedure calls	Main tick loop	Developer

Database Writes	Persist logs and metrics	SQL transactions	End of tick	Developer
-----------------	--------------------------	------------------	-------------	-----------

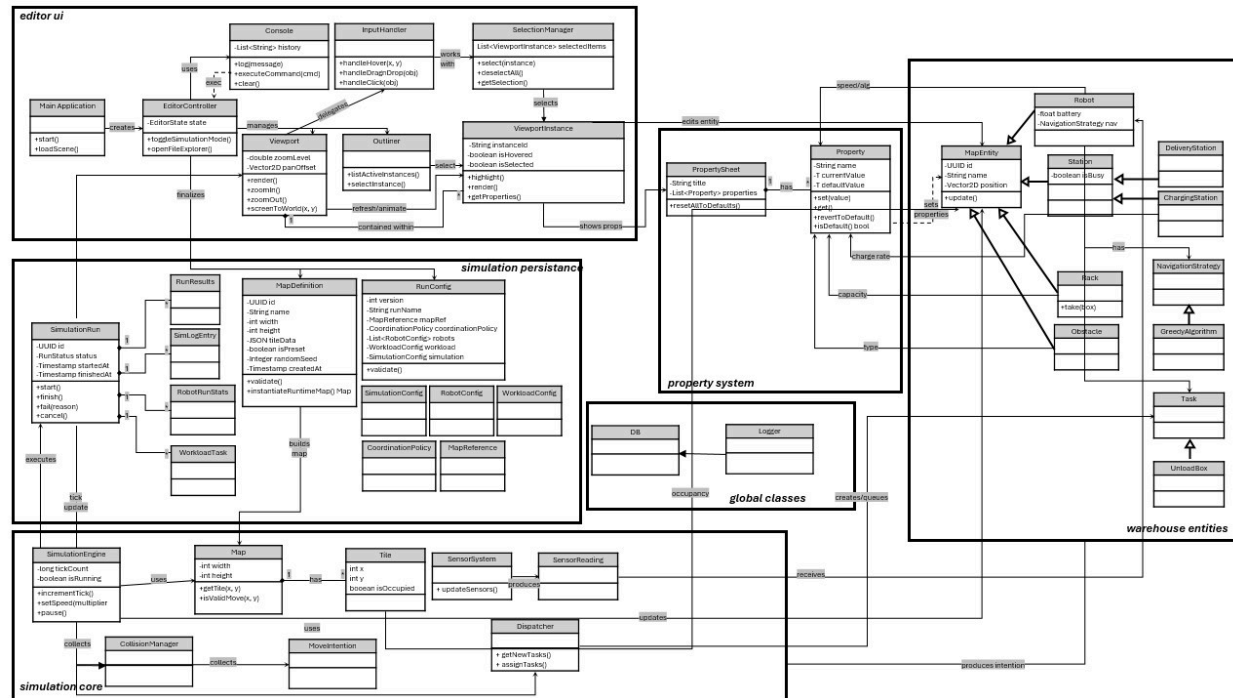
Evolution Phase

Title	Purpose	Technology / Structure	Location	Owner
Algorithm Extension	Introduce new navigation strategies	Pluggable algorithm interfaces	Robot logic	Developer
Policy Extension	Add new coordination mechanisms	Policy interfaces	Coordination module	Developer
Schema Extension	Support new metrics and outputs	JSONB fields	Database layer	Developer

These procedural structures ensure the system remains testable during development, deterministic during execution, and extensible over time as new algorithms, policies, and metrics are introduced.

3.3 UML Domain Model

Figure 3.3.1 – UML Domain Model ([View here in PowerPoint](#))



The UML Domain Model represents a modular, layered architecture that separates authoring, persistence, simulation logic, and visualization concerns. The model is organized into six major subsystems: Editor UI, Simulation Persistence, Simulation Core, Warehouse Entities, supporting Global Classes, and Property System. This separation ensures deterministic execution, maintainability, and extensibility while supporting both interactive editing and reproducible simulation runs.

3.3.1 Editor UI Subsystem

The Editor UI subsystem provides the authoring environment in which users construct maps, configure robots, and modify simulation parameters before execution.

The MainApplication initializes the JavaFX environment and delegates control to the EditorController, which manages transitions between edit mode and simulation mode. The EditorController coordinates UI components including:

- Viewport – Responsible for rendering entities, handling zoom/pan transformations, and maintaining screen-to-world coordinate conversions.
- Outliner – Displays active entities within the scene and supports selection.
- SelectionManager – Maintains the current set of selected ViewportInstance objects.
- InputHandler – Processes user input events such as clicks and drag-and-drop actions.

- Console – Maintains command history and allows execution of user commands.

Each visual element in the editor is represented by a `ViewportInstance`, which references a unique domain entity (via `instanceld`) and supports selection state and rendering behavior. This abstraction cleanly separates UI visualization from simulation entities.

The editor does not directly modify the simulation engine during runtime. Instead, it produces configuration data that is later consumed by the simulation persistence layer.

3.3.2 Simulation Persistence Subsystem

The Simulation Persistence subsystem bridges authoring and execution by defining configuration structures and managing runtime results.

Key classes include:

- `MapDefinition` – Stores static map data (dimensions, seed, name) and validates structural correctness before instantiating a runtime Map.
- `RunConfig` – Aggregates simulation parameters including coordination policy, workload configuration, robot configuration, and map references.
- `SimulationRun` – Represents a single execution instance and tracks lifecycle state (start, pause, cancel, finish).
- `RunResults`, `RobotRunStats`, `SimLogEntry`, and `WorkloadTask` – Store metrics and historical data for analysis and replay.

This subsystem ensures reproducibility: simulations are instantiated from validated configuration objects rather than live UI state. The `instantiateRuntimeMap()` method in `MapDefinition` cleanly separates configuration data from executable domain objects.

3.3.3 Simulation Core Subsystem

The Simulation Core contains the deterministic execution engine and implements the per-tick pipeline described in Critical Section 1.

The central class is:

SimulationEngine

Responsibilities include:

- Maintaining global tick counter
- Managing execution state (`isRunning`)
- Advancing simulation via `incrementTick()`
- Applying speed modifiers
- Coordinating subsystems each tick

The engine interacts with:

- Map – Stores runtime Tile objects and validates movement legality.
- CollisionManager – Collects and resolves movement conflicts.
- Dispatcher – Assigns tasks to robots.
- SensorSystem – Updates environmental awareness.
- MovementIntent – Represents proposed robot actions for each tick.

The Map owns multiple Tile objects, each storing spatial coordinates and occupancy state. This ensures that at most one robot occupies a tile at the end of each tick, preserving safety constraints. There is no RobotManager class since it's more of a module that encompasses how the robots will be managed, with whom they will be managed by different classes as mentioned previously like Dispatcher, SensorSystem, MovementIntent, etc.

3.3.4 Warehouse Entities Subsystem

The Warehouse Entities subsystem models all active physical objects in the simulation.

Each Robot has:

- Unique identifier
- Position (Vector2D)
- Battery level
- NavigationStrategy
- State (e.g., idle/busy)
- Task

Robots do not directly commit movement. Instead, they produce movement intentions that are validated and resolved by the Simulation Engine, reinforcing centralized control.

Supporting Entities:

- Rack
- Obstacle
- DeliveryStation
- ChargingStation

These entities define interaction targets for robots. The inheritance structure around NavigationStrategy (e.g., GreedyAlgorithm) supports extensible path-planning approaches without modifying core robot logic.

This design follows the Strategy Pattern, allowing dynamic policy substitution.

3.3.5 Property System

The Property System provides a generic, extensible mechanism for editing and managing configurable attributes across entities.

- PropertySheet aggregates multiple Property objects.
- Each Property includes name, current value, default value, and validation logic.
- The system supports dynamic editing within the UI while preserving type safety.

This abstraction prevents hardcoding of parameters and supports future extensibility (e.g., adding new robot attributes without modifying the UI framework).

3.3.6 Global Classes

Global utility classes such as Logger and DB provide cross-cutting concerns:

- Logging per-tick events
- Recording metrics
- Supporting persistence and offline analysis

These classes are intentionally decoupled from the simulation core to preserve deterministic runtime behavior while enabling comprehensive experimentation and result tracking.

3.4 Module breakdown

This section provides a high-level breakdown of the eight software modules that compose the OpenRobotics system, as reflected in the Specification Document's deployment view (Section 2.2.7) and the deployment environment (Section 2.3). Each module encapsulates a distinct responsibility and interacts with other modules through clearly defined interfaces. Closely related functionality is grouped within the same module where appropriate, consistent with the architectural structures introduced in Section 3.2 and the UML domain model in Section 3.3. This modular decomposition supports separation of concerns, deterministic execution, and extensibility.

3.4.1 Simulation Engine Module

Responsibility:

The Simulation Engine serves as the central coordinator of the system. It controls the deterministic per-tick execution pipeline (CS1), advances global simulation time, and orchestrates interactions between robots, coordination policies, task dispatching, logging, and metrics collection. The Simulation Engine orchestrates coordination policy enforcement (TRAFFIC_RULES and RESERVATION_K, CS4 and CS5) and invokes collision resolution logic via the CollisionManager component (CS2).

Key Interactions:

- Invokes the Dispatcher Module for task assignment at the start of each tick
- Collects robot intentions via the Robot Core Module
- Orchestrates coordination policy enforcement and collision resolution before committing movement
- Triggers Database Client for logging and metric updates at the end of each tick
- Queries the Map Module for spatial validation

This module enforces the fixed tick order that underpins fairness, reproducibility, and correctness across all simulation runs.

3.4.2 Sensors Module

Responsibility:

The Sensors Module provides robots with environmental awareness, enabling them to perceive their local surroundings and detect obstacles. This module supports partial observability by limiting each robot's knowledge to a configurable sensor range, which influences navigation decisions and collision avoidance behavior.

Key Interactions:

- Queries the Map Module to determine visible tiles and obstacles within sensor range

- Provides sensor data to the Robot Core Module for navigation planning
- Updates sensor views at the beginning of each tick before robots compute intentions

The Sensors Module enables robots to make informed navigation decisions while maintaining the simulation's deterministic behavior through consistent sensor update ordering.

3.4.3 Robot Core Module

Responsibility:

The Robot Core Module represents individual robots, maintaining state such as position, energy level, current task, and operational status (Idle, Moving, Loading, Unloading, Charging). This module implements robot state machines and energy models, tracking battery consumption during movement and task execution.

Key Interactions:

- Receives task assignments from the Dispatcher Module
- Queries the Sensors Module for environmental awareness
- Computes movement intentions using navigation algorithms from the Algorithms Module
- Submits intentions to the Simulation Engine for validation and coordination
- Updates the robot state following committed movements

Robot decision logic is intentionally lightweight, as the Simulation Engine handles conflict resolution and coordination to ensure deterministic behavior.

3.4.4 Dispatcher Module

Responsibility:

The Dispatcher Module manages workload tasks and is responsible for task generation, assignment, and reassignment (see CS3 - Critical Section 3). It ensures tasks are assigned exactly once and only to eligible robots, maintaining task queues and tracking task status throughout the simulation.

Key Interactions:

- Generates new tasks based on workload configuration at configurable tick intervals
- Maintains task queues (pending, assigned, completed, failed)
- Assigns tasks to eligible robots at the beginning of each tick
- Requeues tasks during deadlock recovery or robot failure (see CS6 - Critical Section 6)
- Logs task assignment and reassignment events via the Database Client

Centralizing task management simplifies correctness guarantees and supports recovery mechanisms while ensuring fairness in workload distribution.

3.4.5 Algorithms Module

Responsibility:

The Algorithms Module implements pluggable navigation strategies used by robots to plan movement toward task goals. Supported navigation strategies include GREEDY, BUG, and RTA_STAR (see CS7 - Critical Section 7 for the GREEDY strategy). These strategies operate under the constraints imposed by the active coordination policy and sensor information.

Key Interactions:

- Invoked by the Robot Core Module to compute intended actions based on sensor data and task goals
- Queries the Map Module for structural information (walkable tiles, intersections, special locations)
- Returns movement intentions that are subject to validation and coordination by the Simulation Engine

The modular design allows additional navigation algorithms to be introduced without modifying the simulation core, supporting experimentation and comparative analysis of different navigation approaches.

3.4.6 Map Module

Responsibility:

The Map Module loads and manages the warehouse layout, including grid dimensions, walkable tiles, obstacles, intersections, and special locations such as packing stations, docks, and charging stations. Maps can be loaded from preset configurations bundled with the application or generated procedurally with configurable random seeds to support experimental variation.

Key Interactions:

- Provides spatial constraints to the Simulation Engine, Sensors Module, and navigation algorithms
- Validates movement legality (tile walkability, bounds checking)
- Supplies location information for task pickup and drop-off points
- Remains read-only during simulation execution to ensure reproducibility

This module ensures consistent interpretation of the warehouse environment across all subsystems and supports the deterministic behavior required for experimental validity.

3.4.7 Database Client Module

Responsibility:

The Database Client Module handles all persistence concerns, including establishing connections to the PostgreSQL database (as described in Section 4), logging simulation events, storing per-robot and run-level metrics, and supporting replay and offline analysis. This module executes SQL queries to write simulation data to the six database tables (maps, simulation_runs, run_results, sim_logs, robot_run_stats, run_workload_tasks).

Key Interactions:

- Records per-tick events and metrics to the sim_logs table during simulation execution
- Writes simulation configuration to the simulation_runs table at run creation
- Stores aggregated results to run_results, robot_run_stats, and run_workload_tasks tables upon completion
- Supports post-run analysis, historical data retrieval, and CSV/JSON export functionality

Persistence is isolated from core simulation logic to avoid interfering with deterministic execution while still providing comprehensive experimental data for analysis.

3.4.8 UI Module

Responsibility:

The UI Module provides a JavaFX-based graphical interface for configuring simulations, controlling execution, visualizing robot movement in real time, and inspecting results. This module implements the four main screens defined in the Specification Document (Section 4.1): Welcome Screen, Setup Screen, Simulation Screen, and Results Screen.

Key Interactions:

- Communicates with the Simulation Engine via control commands (start, pause, resume, stop)
- Loads and saves configuration files (JSON format) for reproducible experiments
- Queries the Database Client for historical run data and metrics visualization
- Displays real-time simulation state by observing robot positions and task progress
- Allows users to export results via the Database Client's export functionality
- Provides simulation speed controls (e.g., x1, x2, x3) during execution to allow users to adjust playback rate.

The UI is decoupled from the simulation core through well-defined APIs, enabling independent evolution of presentation and logic without affecting simulation determinism.

3.4.9 File Loader Module

Responsibility:

The File Loader Module is responsible for parsing, validating, and constructing simulation configuration data from external JSON files prior to execution. This module enables reproducible simulations by transforming user-authored configuration files (generated via the UI Setup Screen and configuration files) into domain objects used to initialize the Simulation Engine. It ensures that map layouts, robot initial states, workload parameters, and simulation settings are syntactically correct, semantically valid, and internally consistent before runtime begins.

The File Loader performs structural validation (required fields, correct data types, schema compliance) as well as logical validation (no overlapping robots, valid tile coordinates, robots placed on walkable tiles, parameter ranges within acceptable bounds). Invalid configurations are rejected before engine initialization, preventing undefined behavior during execution.

Key Interactions:

- Parses JSON configuration files produced by the UI Module (Setup Screen & Config Files)
- Constructs domain objects, including Map, Tile, Robot, Dispatcher configuration, and Simulation Parameters
- Performs pre-run validation checks prior to Simulation Engine instantiation
- Returns a validated SimulationConfig object to the Simulation Engine
- Provides structured error feedback to the UI Module when configuration issues are detected
- Supports loading previously saved experimental scenarios for replay or parameter comparison

The File Loader operates strictly during the transition from Setup Mode to Simulation Mode and does not participate in per-tick execution. By isolating configuration parsing and validation outside the Simulation Engine, the system preserves deterministic behavior during runtime while maintaining flexibility in scenario creation and experiment reproducibility.

Summary

Together, these nine modules form a cohesive architecture in which centralized control (Simulation Engine) ensures determinism, while modular decomposition enables extensibility and maintainability. This breakdown aligns directly with the deployment environment described in Section 2.3, the code structures defined in Section 3.2, and the UML domain model presented in Section 3.3. The module interactions support the per-tick execution pipeline (CS1) and enable safe, reproducible warehouse simulations. Each module provides the foundation for the critical sections and mitigation models analyzed in Section 5, ensuring that the system

meets its requirements for fairness, safety, and experimental validity as specified in the Specification Document.

4. Database, File, API Definitions

Section 4 defines how simulation data, configuration parameters, runtime logs, and experimental results are stored, accessed, and exchanged within our system.

The database schema, file definitions, and API contracts presented here are derived from Specification Document, and are described in the sense of the Design Document.

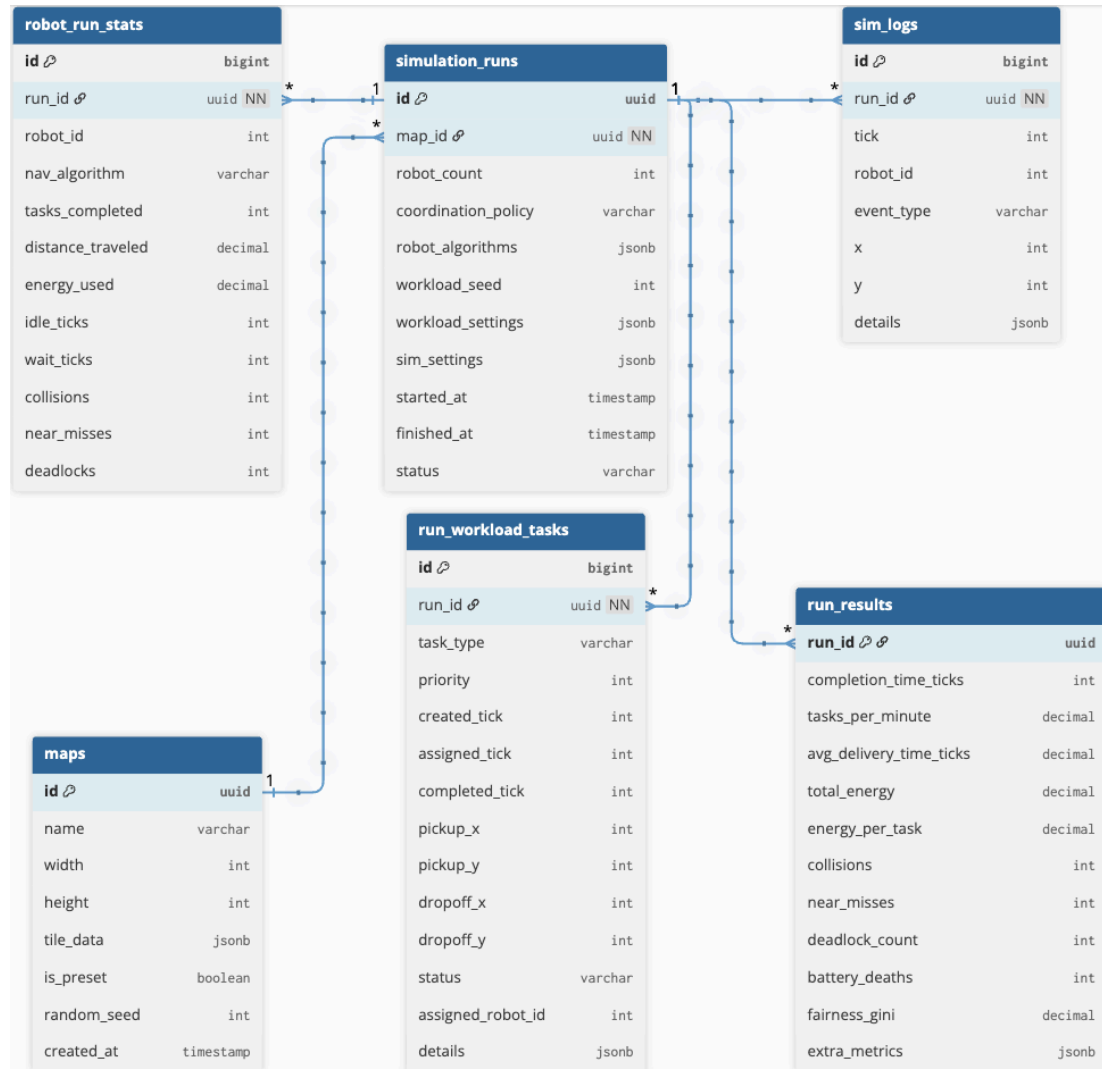
4.1 Database Diagrams

The OpenRobotics database consists of six tables used to persist warehouse maps, simulation run configurations, aggregated experiment results, per-tick simulation logs, per-robot statistics, and workload task records.

Tables included in the database diagram:

1. maps
2. simulation_runs
3. run_results
4. sim_logs
5. robot_run_stats
6. run_workload_tasks

Figure 4.1.1 – Database Diagram



The database is centered around the **simulation_runs** table, which represents a single execution of a simulation experiment. All runtime data, including logs, per-robot statistics, workload tasks, and aggregated results, reference a specific simulation run via foreign keys. This structure ensures isolation between experiments, supports reproducibility, and enables efficient historical analysis of simulation outcomes.

4.2 Database Schemas

4.2.1 maps

Stores warehouse maps, including dimensions and tile layout data.

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	UUID	36 chars	Primary key	Not null, auto-generated
name	String	100 chars	Index (optional)	Not null
width	Int	4 digits	–	> 0
height	Int	4 digits	–	> 0
tile_data	JSONB	–	–	Not null, valid map JSON structure
is_preset	Boolean	1	–	Default false
random_seed	Int	10 digits	–	Nullable (null for presets)
created_at	Timestamp	–	Sort DESC	Default now

4.2.2 simulation_runs

Represents one simulation execution (one experiment run) and stores its configuration.

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	UUID	36 chars	Primary key	Not null, auto-generated
map_id	UUID	36 chars	Foreign key → maps(id)	Not null
robot_count	Int	4 digits	–	≥ 1
coordination_policy	String	30 chars	–	Must be TRAFFIC_RULES or RESERVATION_K

robot_algorithms	JSONB	–	–	Not null (robotId → algorithm mapping)
workload_seed	Int	10 digits	–	Not null
workload_settings	JSONB	–	–	Not null
sim_settings	JSONB	–	–	Not null
started_at	Timestamp	–	Sort DESC	Default now
finished_at	Timestamp	–	–	Nullable
status	String	15 chars	Index	Must be one of : CREATED, RUNNING, COMPLETED, FAILED, CANCELLED

4.2.3 run_results

Stores aggregated (run-level) outcome metrics for a simulation run.

Field Name	Type	Length	Key, Primary, Sort	Constraints
run_id	UUID	36 chars	Primary key + FK → simulation_runs (id)	Not null, cascade delete
completion_time_ticks	Int	10 digits	–	Nullable if failed/cancelled
tasks_per_minute	Decimal	–	–	≥ 0
avg_delivery_time_ticks	Decimal	–	–	≥ 0
total_energy	Decimal	–	–	≥ 0
energy_per_task	Decimal	–	–	≥ 0

collisions	Int	10 digits	–	Default 0, ≥ 0
near_misses	Int	10 digits	–	Default 0, ≥ 0
deadlock_count	Int	10 digits	–	Default 0, ≥ 0
battery_deaths	Int	10 digits	–	Default 0, ≥ 0
fairness_gini	Decimal	–	–	Typically 0..1 (enforce in app layer)
extra_metrics	JSONB	–	–	Default {}

4.2.4 sim_logs

Stores per-tick simulation log events for replay/debugging and analysis.

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	BigSerial	–	Primary key	Auto-increment
run_id	UUID	36 chars	Foreign key → simulation_runs(id)	Not null
tick	Int	10 digits	Sort ASC	≥ 0
robot_id	Int	10 digits	Index (optional)	Nullable for global events
event_type	String	30 chars	Index	Not null (enum enforced in app)
x	Int	10 digits	–	Nullable
y	Int	10 digits	–	Nullable
details	JSONB	–	–	Default {}

4.2.5 robot_run_stats

Stores per-robot aggregated metrics for a specific run.

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	BigSerial	–	Primary key	Auto-increment
run_id	UUID	36 chars	Foreign key → simulation_runs(id)	Not null
robot_id	Int	10 digits	Unique per run	Unique(run_id, robot_id)
nav_algorithm	String	15 chars	–	Must be one of the defined algorithms
tasks_completed	Int	10 digits	–	Default 0
distance_traveled	Decimal	–	–	≥ 0
energy_used	Decimal	–	–	≥ 0
idle_ticks	Int	10 digits	–	≥ 0
wait_ticks	Int	10 digits	–	≥ 0
collisions	Int	10 digits	–	≥ 0
near_misses	Int	10 digits	–	≥ 0
deadlocks	Int	10 digits	–	≥ 0

4.2.6 run_workload_tasks

Stores the workload tasks spawned/used in a run (including assignment and completion ticks).

Field Name	Type	Length	Key, Primary, Sort	Constraints
id	BigSerial	—	Primary key	Auto-increment
run_id	UUID	36 chars	Foreign key → simulation_runs(id)	Not null
task_type	String	15 chars	—	PICK_DELIVER or REPLENISH
priority	Int	4 digits	—	Default 0
created_tick	Int	10 digits	Sort ASC	≥ 0
assigned_tick	Int	10 digits	—	Nullable
completed_tick	Int	10 digits	—	Nullable
pickup_x	Int	10 digits	—	Not null
pickup_y	Int	10 digits	—	Not null
dropoff_x	Int	10 digits	—	Not null
dropoff_y	Int	10 digits	—	Not null
status	String	12 chars	Index	CREATED/ASSIGNED/COMPLETED/CANCELLED/FAILED
assigned_robot_id	Int	10 digits	—	Nullable

details	JSONB	—	—	Default {}
---------	-------	---	---	------------

4.3 File Definitions

This section describes the external file formats used by OpenRobotics for configuration, reproducibility, and data export. These files allow experiments to be saved, shared, and analyzed independently of the database and user interface.

4.3.1 Map Definition Files (JSON)

Map definition files describe the warehouse layout used by a simulation, including grid dimensions and tile-level information.

Maps are stored as JSON files containing the warehouse width, height, and a serialized grid representation. These files are used to define both preset maps and procedurally generated maps. When a simulation is configured, the selected map file is parsed into memory and used to initialize the simulation environment.

Usage:

- Read when configuring or starting a simulation
- Not modified during simulation execution
- May be reused across multiple simulation runs

4.3.2 Simulation Configuration Files (JSON)

Simulation configuration files store all parameters required to reproduce a simulation experiment.

Configuration files are stored in JSON format and include parameters such as robot count, navigation algorithms, coordination policy, workload settings, and random seeds. These files allow experiments to be saved, reloaded, and shared without relying on UI state or database records.

Usage:

- Created when a user saves an experiment configuration
- Read when loading or starting a simulation
- Ensures deterministic and reproducible execution

4.3.3 Result Export Files (CSV / JSON)

Result export files allow simulation outcomes to be analyzed outside the OpenRobotics application.

Simulation results and metrics can be exported in CSV or JSON format for use in external tools such as spreadsheets or data analysis software. Exported data includes aggregated run metrics, per-robot statistics, and optionally task-level or log-level data.

Usage:

- Generated after a simulation completes
- Read-only output
- Used for offline analysis, reporting, or comparison across runs

4.3.4 Logging and Temporary Files

These files support debugging and runtime inspection during development and execution.

Any temporary or debug files generated by the simulator are local to the user's machine and are not required for normal operation. On the other hand, persistent logging and analysis data is stored in the database rather than in local files.

Usage:

- Optional and environment-dependent
- Not required for experiment reproducibility

4.4 API Definitions

This section describes the application programming interfaces (APIs) used by OpenRobotics to coordinate interactions between the user interface, simulation engine, and persistence layer. APIs are grouped by responsibility to clearly separate configuration management, simulation lifecycle control, result retrieval, and data export. Keep in mind that there are actually no endpoints involved for a Maven Java/JavaFX project; the frontend and backend communicate with direct object instantiations and method calling.

4.4.1 Map APIs

Map APIs provide access to warehouse map definitions used during simulation setup. These APIs allow the application to retrieve available preset maps, load map metadata, and associate maps with simulation runs. Maps are treated as read-only configuration data once a simulation begins.

Usage:

- Used during experiment configuration
- Accessed by the UI when selecting or previewing maps
- Not modified during simulation execution

4.4.2 Simulation Run APIs

Simulation run APIs manage the lifecycle of simulation experiments. These APIs are responsible for creating new simulation runs, starting execution, updating run status, and recording the run's status (completion or failure). They serve as the primary interface between the UI and the simulation engine.

Usage:

- Invoked when starting or stopping simulations
- Polled by UI to track execution progress
- Write simulation metadata and status to persistent storage

4.4.3 Results and Metrics APIs

Results APIs provide access to aggregated simulation outcomes and performance metrics. These APIs expose metrics, robot statistics, and summary data required for analysis. Result data is generated by the simulation engine and retrieved by UI after simulation completion.

Usage:

- Used by the results dashboard
- Supports comparison across multiple runs
- Read-only after simulation completion

4.4.4 Logging and Replay APIs

Logging APIs support simulation replay, debugging, and detailed timeline analysis.

These APIs provide access to simulation logs: robot movements, task events, and collision events. These APIs are designed to support filtered or incremental retrieval since the user can potentially get a large volume of log data.

Usage:

- Used during replay and visualization
- Accessed for debugging and analysis
- Read-only during normal operation

4.4.5 Export APIs

Export APIs allow simulation data to be extracted and saved on user's local machine.

These APIs generate CSV or JSON exports of simulation results and metrics, potentially some detailed data. Export functionality enables offline analysis without requiring direct database access.

Usage:

- Invoked after simulation completion
- Generates read-only output files
- Used for reporting and external tools

5. Critical Sections

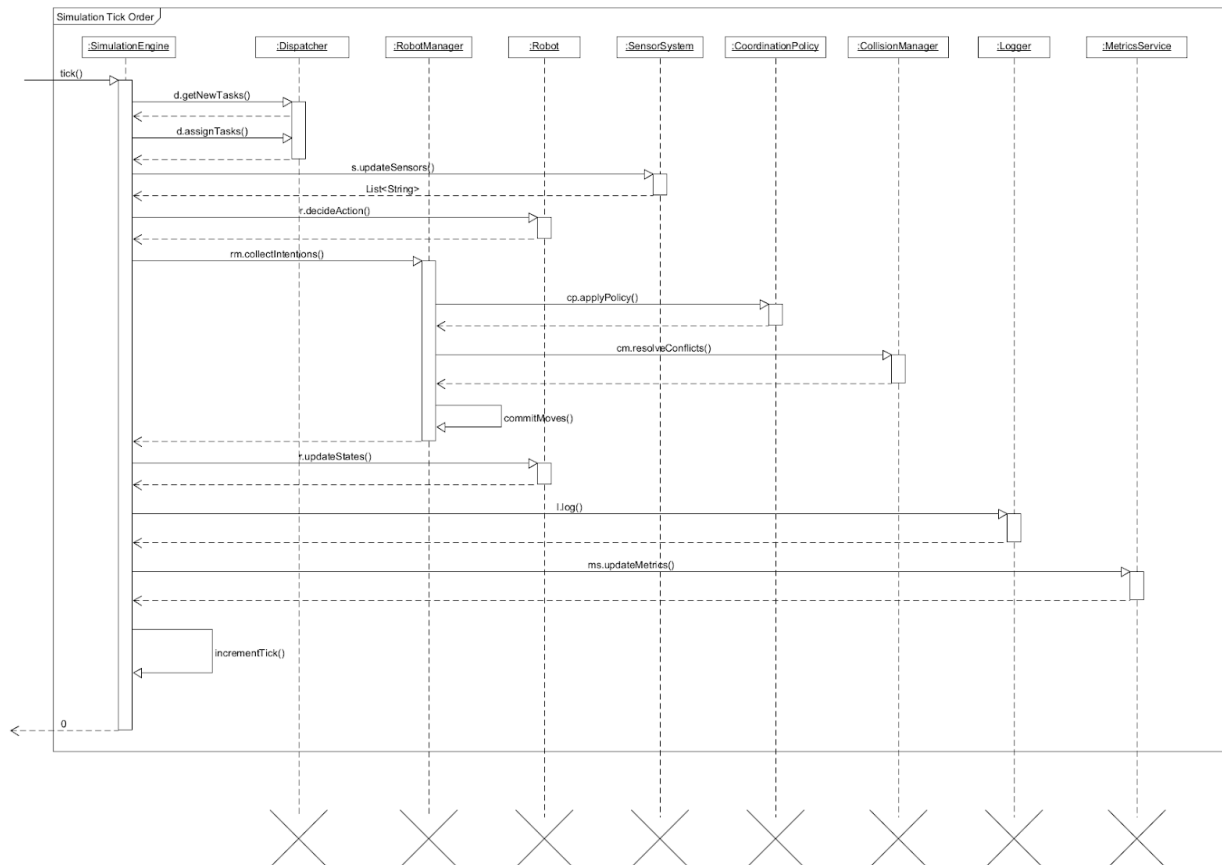
An analysis of the critical sections and mitigation models used to model the solution.

5.1 Risk Analysis

Item	Module	Location	Description	Modelling Tool
CS1: Tick Order	Simulation Engine	Main/Simulation class	Defines the fixed order of operations each tick to ensure fairness and reproducibility.	UML Sequence Diagram
CS2: Move Intentions	Collision Manager	Collision Manager Class/RobotManager	Defines how no two robots end a tick step in the same tile and prevents unsafe interactions.	UML Activity Diagram
CS3: Task Assignment	Dispatcher	Dispatcher class	Ensures tasks are assigned once, not lost/duplicated, and assigns only to eligible robots.	UML Activity Diagram
CS4: Coordination Policy	Simulation Engine	CoordinationPolicy, SimulationEngine, and Robot classes	Defines traffic rules for moving through aisles, intersections, and pick-up/drop-off areas.	UML Activity Diagram
CS5: Tile Reservation	Simulation Engine	CoordinationPolicy, SimulationEngine, and Robot classes	Defines rules for the reservation of tiles by robots when moving throughout the warehouse map.	UML Activity Diagram
CS6: Deadlock Detection	Dispatcher	Dispatcher class and	Detects robots stuck for T amount of ticks.	UML Activity Diagram
CS7: Greedy Strategy	Simulation Engine	Robot class	Defines a navigation strategy for robots where they move towards their goals by making the most greedy choice to reduce manhattan distance.	UML Activity Diagram

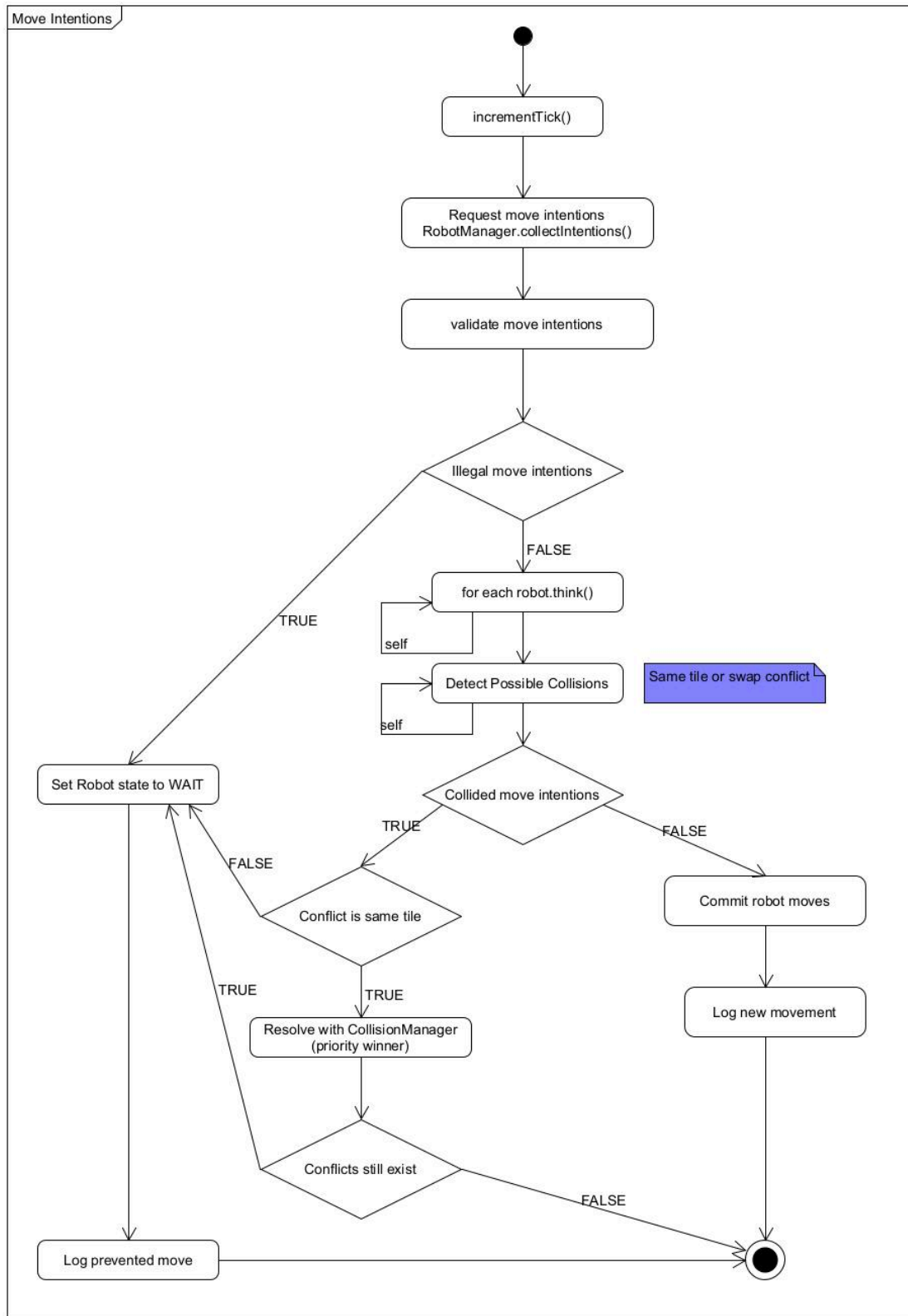
5.2 Mitigation Models

Figure 5.2.1 – CS1: Tick Order



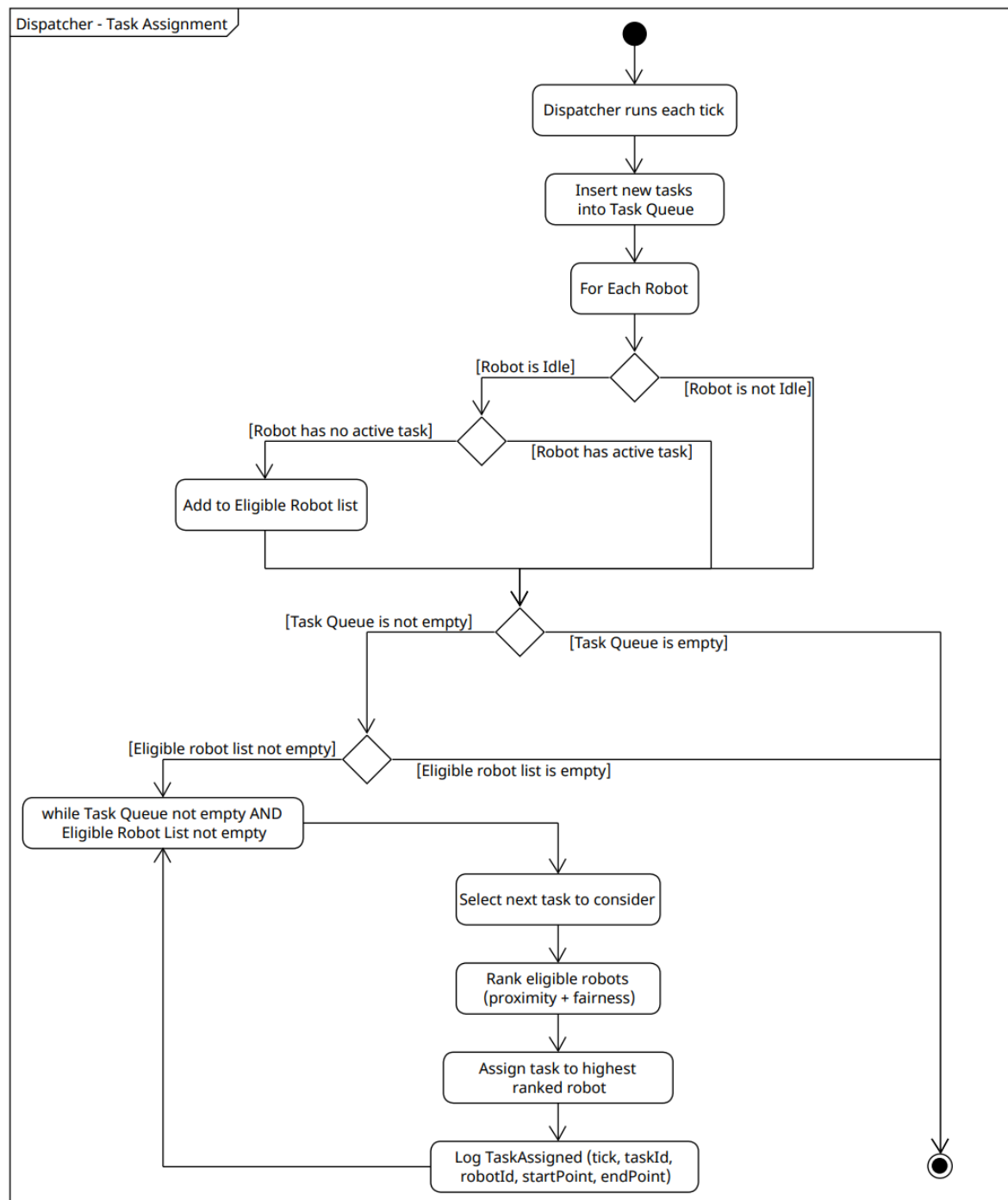
The above figure shows the per-tick simulation update pipeline controlled by the SimulationEngine. At the start of each tick, the engine checks whether the workload is complete; otherwise, it generates new requests and updates the task queue. The dispatcher assigns tasks to eligible robots, after which each robot updates its sensor view, selects an intended action, and submits its move intention. The system then applies the active coordination policy and detects conflicts, resolving them using deterministic priority rules when necessary. Once a safe set of actions is finalized, the engine commits all movements simultaneously, updates robot states and timers, records events, updates live metrics and visualizations, and increments the tick counter. This fixed order ensures fairness, reproducibility, and consistent measurement of system performance across runs.

Figure 5.2.2 – CS2: Move Intentions



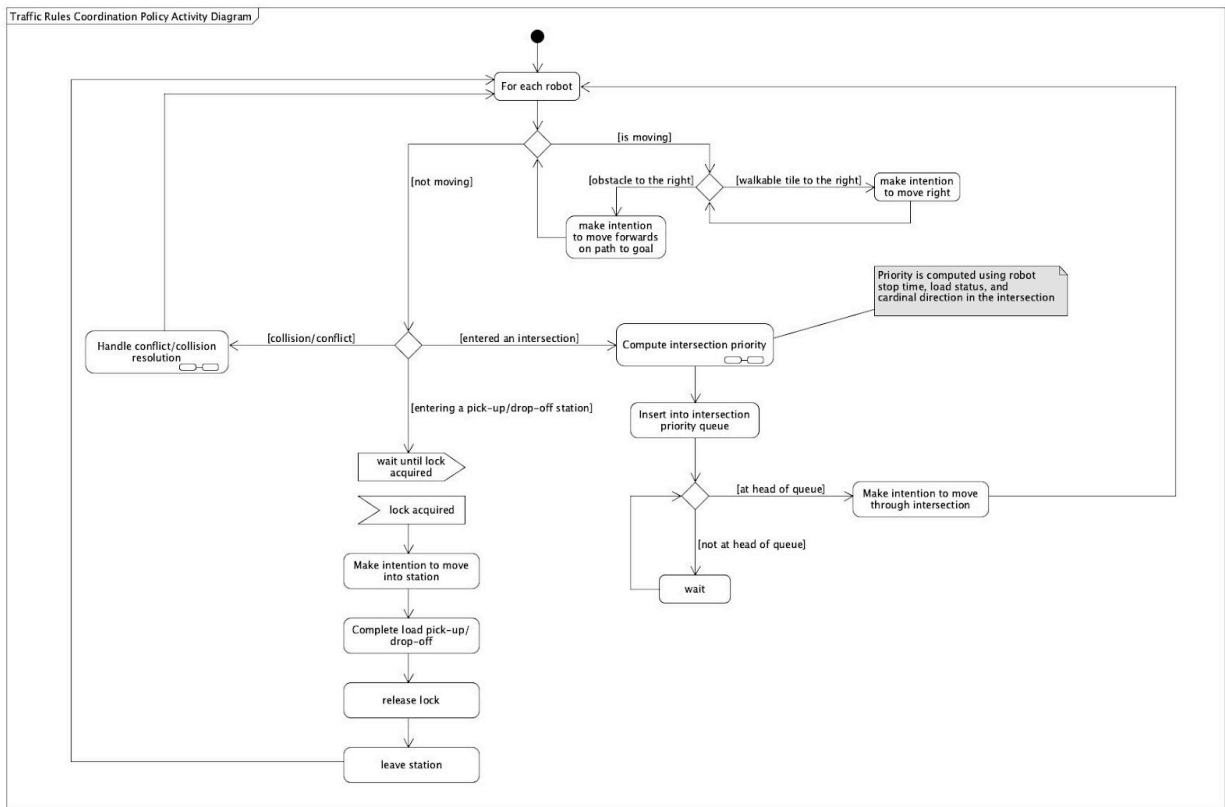
The above figure shows the move-intention and conflict-resolution pipeline executed during each simulation tick. At the start of the tick, the Simulation Engine requests all robots to submit their intended actions, which may include moving to an adjacent walkable tile, waiting, or performing a non-movement action such as loading or unloading. The engine then validates each intention to ensure it is legal with respect to map bounds, walkability, one-tile-per-tick movement constraints, and the currently active coordination policy. After validation, the system analyzes the set of proposed moves to detect potential conflicts, including same-tile conflicts and swap conflicts between robots. When conflicts are detected, the CollisionManager applies a deterministic priority and tie-breaking policy to select which robot, if any, is allowed to proceed, while all other robots involved are forced to wait for the tick. Once a conflict-free set of moves is produced, the engine commits the approved movements simultaneously, updates robot states, logs prevented or successful movements, and records congestion and waiting metrics. This process ensures that no two robots can occupy the same tile at the end of a tick while maintaining fairness and reproducibility across runs.

Figure 5.2.3 – CS3: Task Assignment



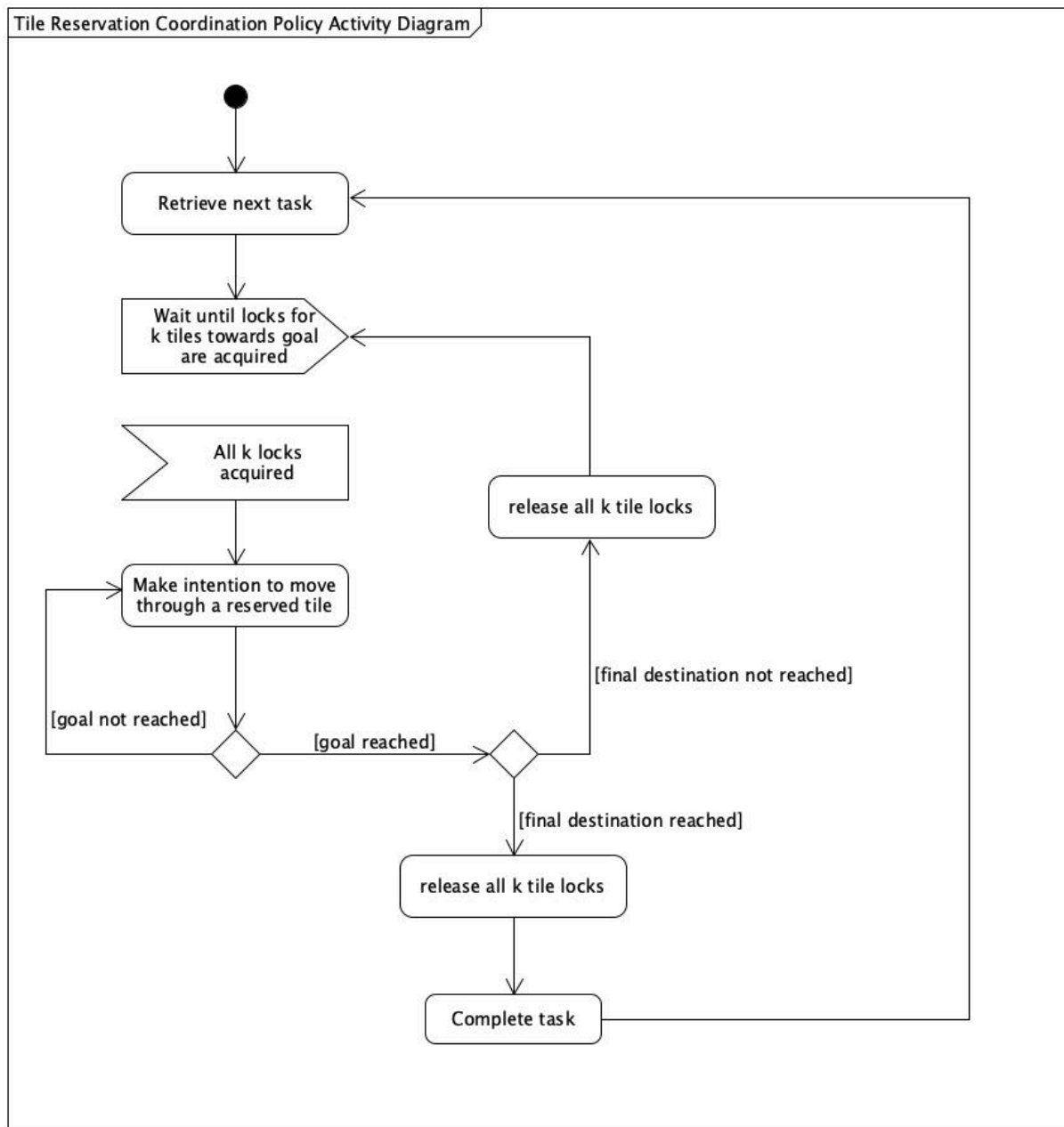
The above figure shows how the dispatcher assigns tasks at the start of each tick after new requests are added to the Task Queue. The dispatcher first builds an eligible robot set (e.g., Idle, not disabled, and not already assigned a task), then selects tasks and ranks feasible robots using proximity and fairness (no robot occupies all the tasks). Once selected, the task is assigned to exactly one robot, the robot is removed from the eligible list, and the assignment is logged for later analysis.

Figure 5.2.4 – CS4: Traffic Rules Coordination Policy



The above figure shows a diagram that models traffic coordination. Intersection coordination is controlled by a priority queue. When a robot enters an intersection, it computes a priority based on stop time, load status, and approach direction, then inserts itself into the queue. The robot waits until it reaches the head of the queue before proceeding through the intersection, ensuring mutual exclusion and deterministic resolution of conflicts. Pick-up and drop-off stations are handled separately using locks to guarantee exclusive access.

Figure 5.2.5 – CS5: Tile Reservation



The above figure models tile-based movement protected by multi-tile locking. Before moving, a robot retrieves its next task and waits until the locks for the next k tiles along its path are acquired. The robot then moves through the reserved tiles and releases locks as it progresses or upon reaching the final destination, ensuring safe navigation while preventing collisions and deadlock during path execution.

Figure 5.2.6 – CS6: Deadlock Detection

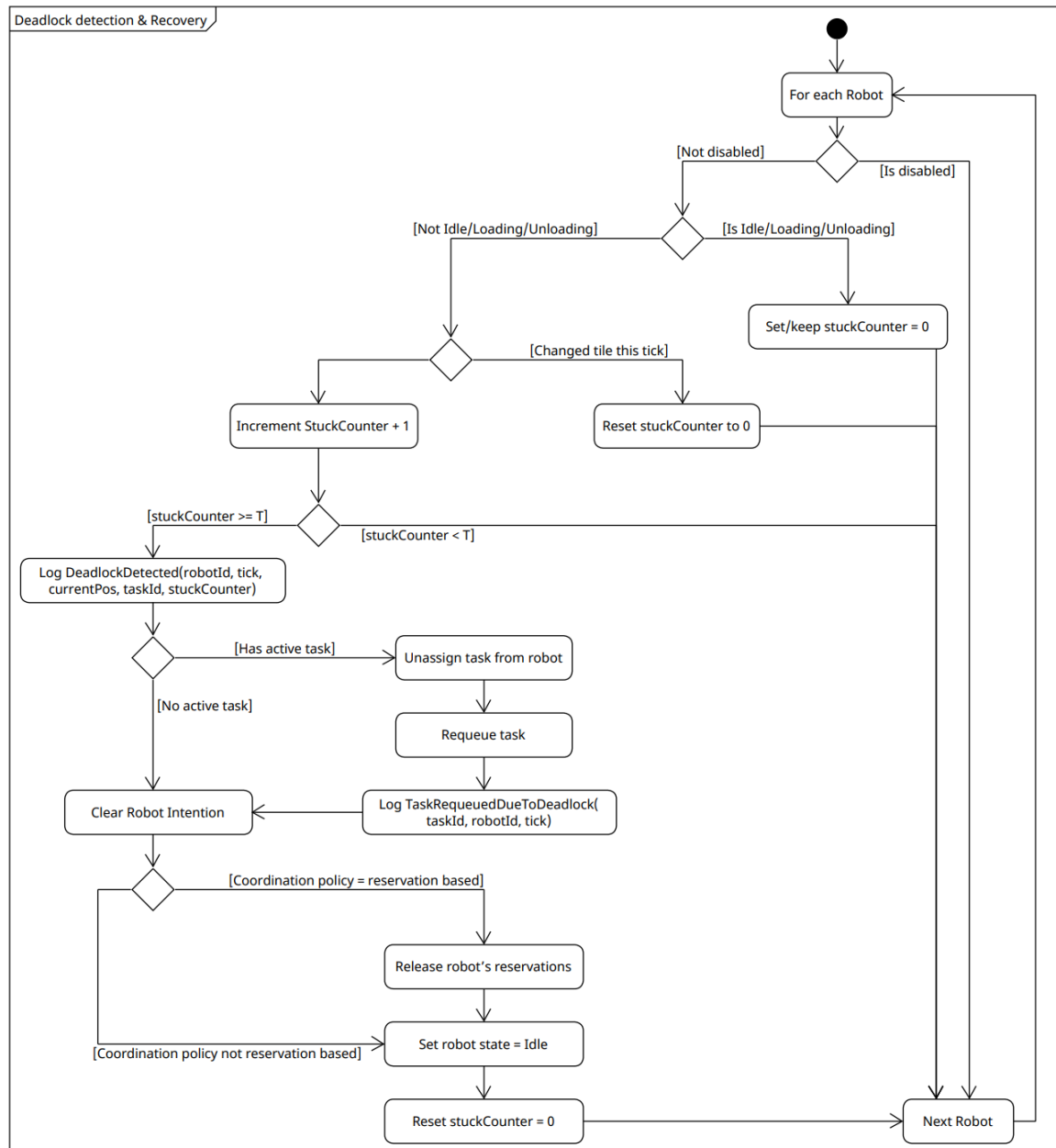
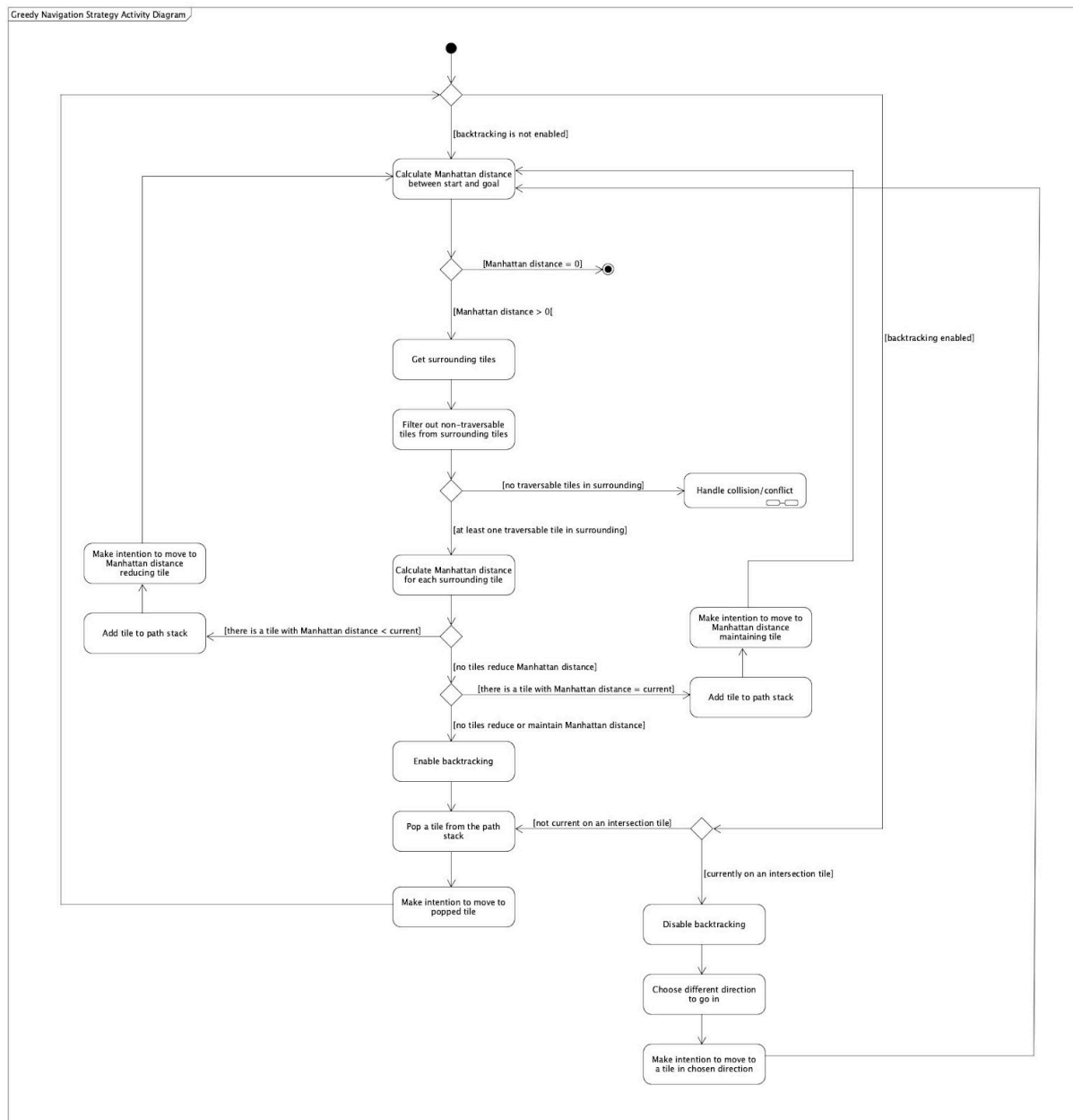


Figure 5.2.6 shows the dispatcher's deadlock monitoring that runs at the end of each tick, after movements are committed, so progress can be measured. For each robot, the diagram resets the stuck counter when the robot is Idle/Loading/Unloading or when it changes tiles, and increments the counter only when the robot is expected to move but doesn't. When the counter reaches T, the recovery branch is triggered: the system logs a DeadlockDetected event, requeues the task if the robot owned one, clears the robot's plan (and releases reservations if applicable), then resets the robot to Idle before resetting the stuck counter. This makes deadlocks fixable without duplicating tasks and ensures deadlocks and recovery are logged for analysis.

Figure 5.2.7 – CS7: Greedy Strategy



The above figure outlines a greedy navigation strategy for individual robots that prioritizes reducing the Manhattan distance to a goal. The process begins by checking if backtracking is enabled; if not, the system calculates the distance to the goal and identifies traversable surrounding tiles. If a tile exists that reduces the Manhattan distance, it is added to the path stack and a move intention to it is set. If no reducing tile is available but a tile maintains the current distance, that tile is used instead. When no tiles can reduce or maintain the distance, the system enables backtracking and backtracks to the last intersection tile to try a different direction of movement.

Appendix A: Teams Contribution Table

Student Name & Student ID	Assigned Section	Contribution
Muhammad Sohail, 261142698 (Backend dev)	5	CS4, CS5, CS7, Backend Domain brainstorming, Overall Formatting & Review
Filip Snítíl, 261139844 (Backend dev)	1, 2	All of section 1 and 2
Murad Novruzov, 261164063 (Frontend dev)	4	All of section 4, Overall Review
Badr Zejli, 261239011 (Frontend dev)	3	Storyboard V2 and UML Domain Model
Lounes Azzoun, 261181741 (QA)	3	Overall review and Polishing UML Domain Model
Daniel Blackburn, 261112665 (Backend dev)	5	CS3, CS6, Review of section 5
Dan Moraru, 261227203 (Lead/Backend dev)	5, 2	CS1, CS2, Backend Domain brainstorming and polishing, Overall Review
Behnam Yosufi, 261125449 (QA/Frontend)	3	Section 3.2, 3.4